

GShard: 通过条件计算与自动分片实现巨型模型的扩展性

Dmitry
Lepikhin@lepidhin@google.com

Hyounk Joong
Leehyouklee@google.com

Yuanzhong
Xuyuanzx@google.com

Dehao
C hendehao@google.com

Orhan
Firat@orhanf@google.com

Yanping
Huanghuan@gyp@google.com

Maxim
Krikun@krikun@google.com

诺姆·沙齐尔
noam@google.com

Zhifeng
Chenzhifeng@google.com

摘要

在拥有海量训练数据和计算的许多真实世界机器学习应用中, 神经网络的扩展性对提升模型质量至关重要。尽管这种扩展趋势被认为是获得更好模型质量的可靠途径, 但在此过程中仍面临诸多挑战, 例如计算成本、编程易用性以及并行设备上的高效实现。GShard 是一个模块, 由一组轻量级注解 API 和对 XLA 编译器的扩展构成。它以对现有模型代码做最少改动的方式, 优雅地表达了各种并行计算模式。借助自动分片, GShard 使我们能够将带有稀疏门控专家混合的多语言神经机器翻译 Transformer 模型扩展到超过 6000 亿参数。我们展示了, 这样的巨型模型可以在 2048 个 TPU v3 加速器上于 4 天内高效地完成训练, 在从 100 种语言到英语的翻译任务上, 相比现有技术取得了远优的质量。

1 引言

提升神经网络的扩展性在广泛的机器学习问题上带来显著的质量提升[1, 2, 3, 4, 5, 6]。对于计算机视觉, 增加模型容量已使各种计算机视觉架构在图像分类和检测的准确率上取得更好表现 [7, 8, 9]。类似地, 在自然语言处理领域, 对 Transformer 模型 [10]进行扩展带来了在语言理解任务 [4, 11, 12]、跨语言下游迁移 [4, 13] 以及 (大规模) 多语言神经机器翻译 [14, 15, 16]上的一致提升。这种总体趋势促使近期研究审视在扩展性 [17, 18, 19, 20, 3], 成功中起关键作用的因素, 包括训练数据量、模型规模以及所使用的计算, 正如以往研究所发现的那样。尽管最终模型质量被发现与数据量、计算和模型规模呈幂律关系 [18, 3], 更大的模型所带来的显著质量提升也伴随着各种实际挑战。训练效率 是最重要的之一, 我们将其定义为在与现有最佳系统对比时达到更优模型质量而使用的计算量和训练时间, 但它常常被忽视。

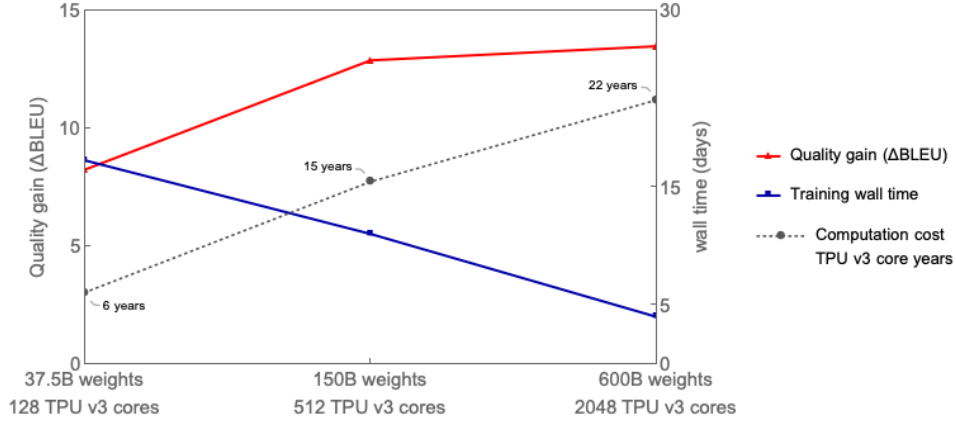


图1: 多语言翻译质量 (相对于双语基线方法的平均 ΔBLEU) 随着专家混合模型规模增大至 600B 而提升, 同时端到端训练成本 (以 TPU v3 核心-年计) 仅呈次线性增长。将模型规模从 37.5B 提升到 600B (16 倍), 计算成本从 6 增至 22 年 (3.6 倍)。达到最佳翻译质量的 600B 参数模型使用 2048 个 TPU v3 核心训练 4 天, 总成本为 22 TPU v3 核心-年。相比之下, 训练全部 100 个双语基线方法模型需要 29 TPU v3 核心-年。我们质量最优的稠密单个 Transformer 模型 (2.3B 参数) 达到 ΔBLEU 6.1, 使用 GPipe [15] 在 2048 个 TPU v3 核心上训练 6 周, 总计 235.5 TPU v3 核心-年。

1.1 扩展性的实际挑战

在此, 我们列举在训练超大规模模型时尤其面临的主要实际挑战, 这些模型的规模比单个加速器内存的容量上限大出若干数量级 (例如, GPU 或 TPU)。

特定架构的模型并行支持 在常用的深度学习框架 (如 TensorFlow [21] 和 PyTorch [22]) 下, 缺乏对高效的模型并行算法的支持。虽然支持基于图划分的朴素模型并行, 但由于网络的顺序依赖和基于梯度的优化, 会导致严重的资源未充分利用。为了高效地扩展现有模型, 用户通常需要投入大量工程工作, 例如将模型代码迁移到特殊的框架 [23, 15]。

计算成本与模型规模之间的超线性缩放 通过增加深度或宽度 [6, 15] 对模型规模进行直接缩放, 通常会导致训练步骤时间至少线性增长。通过在多个设备之间拆分层权重和计算来实现模型并行通常变得必要, 但这会带来网络通信开销和设备利用不足。设备利用不足源于分配不均衡以及底层神经网络的顺序依赖。计算成本与模型规模之间的这种超线性关系并不能仅靠使用更多设备来解决, 从而使训练大规模模型变得不切实际。

面向巨型模型表示的基础设施可扩展性 对于分布在数千台设备上的超大规模模型, 朴素的图表示可能会成为深度学习框架及其优化编译器的瓶颈。例如, 采用算子间划分增加 D 倍的层数, 或采用跨 D 台设备的算子内划分来增大模型维度, 都可能导致图包含 $O(D)$ 个节点。设备之间的通信通道还可能使图规模最多再增加 $O(D^2)$ (例如, 对 `gather` 或 `transpose` 的划分)。这种图规模的增长将导致图构建和编译时间高得不可接受, 对于超大规模模型而言难以实现。

实现分区策略需要付出不小的努力 将模型进行分区以在多台设备上高效运行是具有挑战性的, 因为这需要协调跨设备的通信。对于图级分区, 需要复杂的算法 [15, 24] 来降低开销

这是由分配在不同设备上的图的不同分区之间的顺序依赖所引入的。对于算子级并行，不同被分区的算子依据其语义会采用不同的通信模式，例如，是否需要累加部分结果，或重排数据分片。根据我们的经验，在模型中手动处理这些问题需要大量工作，尤其是考虑到像 TensorFlow 这样的框架拥有大量带有特设语义的算子。无论哪种情况，实现模型分区都会给从业者带来沉重负担，因为更改模型架构将需要相应更改底层设备通信，从而引发连锁反应。

1.2 面向大规模高效训练的设计原则

在本文中，我们通过构建一个拥有 6000 亿参数的序列到序列 Transformer 模型，并在其中加入稀疏门控专家混合 (MoE) 层，展示了如何克服这些挑战；该模型具有次线性计算成本和 $O(1)$ 编译时间。我们在一个多语言机器翻译任务上使用 2048 个 TPU v3 设备训练该模型 4 天，在用单个非集成模型将 100 种语言翻译为英语时，相比以往工作取得了显著更高的翻译质量。我们进行了不同模型规模的实验，发现随着模型变大，翻译质量随之提升，而训练的总壁钟时间仅以相对于模型规模的次线性速度增长，如图1所示。为了构建如此巨大的模型，我们做出了以下关键设计选择。

次线性扩展性 首先，模型架构应被设计为让计算和通信需求相对于模型容量保持次线性。条件计算 [25, 16, 26, 27] 通过针对每个输入激活一个子网络，从而满足训练和推理的效率需求。通过在基于 RNN 的机器翻译和语言模型中加入按位置的稀疏门控专家混合 (MoE) 层 [16] 来扩展容量，可以以次线性计算成本获得最先进结果。因此，我们在第 2 节介绍将 Transformer 架构扩展为包含 MoE 层的方法。

抽象的力量 其次，应将模型描述与划分的实现和优化相分离。这种关注点分离使模型开发者能够专注于网络架构，并能灵活更改划分策略；同时，底层系统执行保持语义的变换并实现高效的并行执行。为此，我们提出了一个模块 GShard，它仅需要用户用划分策略标注模型中的少量关键张量。它由一组用于标注的简单 API，以及 XLA [28] 用于自动并行化的编译器扩展组成。模型开发者可像面对一个具有海量内存和计算容量的单个设备一样编写模型，而编译器会根据这些标注及其自身的启发式规则自动为目标进行计算划分。更多标注示例见第3.2节。

可扩展的编译器 第三，系统基础设施（包括计算表示和编译）必须能够随数千个设备扩展，以实现并行执行。例如，图2展示了将一次点积操作划分到 4 个设备上的两种不同方式（用颜色区分）。请注意，在图2a 所示的常见 MPMD（多程序多数据）方法下，随着设备数量增加，图中的节点数线性增长，使扩展变得更具挑战性。相反，我们为 SPMD（单程序多数据）变换开发了一种编译器技术，它生成一个在所有设备上运行的单一程序，使编译时间保持常数且与设备数量无关，如图2b 所示。我们将在第3.3节更详细地讨论我们的 SPMD 框架。

论文的其余部分组织如下。第2节更详细地描述了包含稀疏门控MoE层的 Transformer 架构。第3节介绍我们的开发模块 GShard。第4节展示了我们专家混合模型在覆盖超过100个语言对的多语言机器翻译任务上的应用。第5节给出了我们实现的性能和内存测量。第6节讨论相关工作。

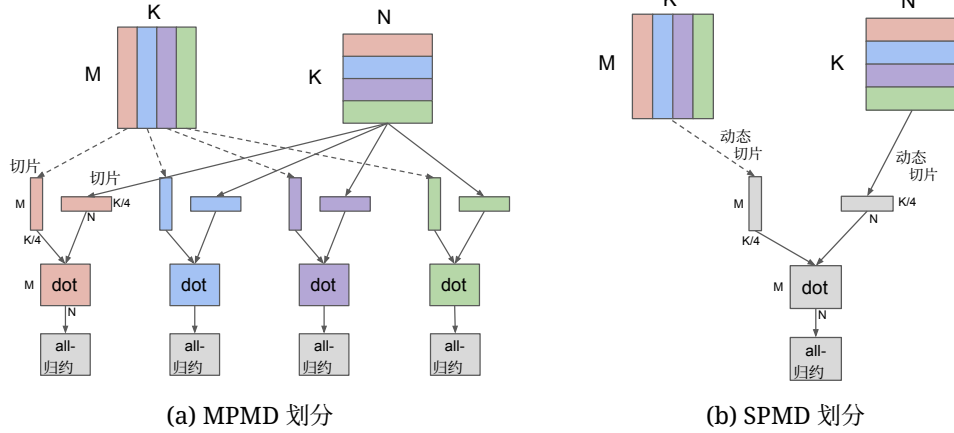


图2: 在 4 个设备上对 Dot 算子 $([M, K] \times [K, N] = [M, N])$ 进行划分时, MPMD 与我们提出的 SPMD 划分的比较。在此示例中, 两个操作数都沿收缩维度 K 进行划分, 每个设备计算本地结果, 并通过 AllReduce 在全局范围内合并。MPMD 划分为每个设备生成单独的算子, 限制了其可扩展性; 而 SPMD 划分生成一个在所有设备上运行的程序。请注意, 我们的 SPMD 划分的编译时间与所用设备数量无关。

2 模型

2.1 Transformer 架构的稀疏扩展性

Transformer [10] 架构已广泛用于自然语言处理。它已成为许多序列到序列任务（如机器翻译）的事实标准。Transformer 架构使用两个计算模块：编码器和解码器，二者均通过堆叠多个 Transformer 层实现。Transformer 编码器层由两个连续的层组成，即一个自注意力层，随后是一个逐位置前馈层。解码器还增加了第三个交叉注意力层，用于关注编码器输出。我们通过条件计算对 Transformer 架构进行稀疏缩放：在编码器和解码器中，将每隔一个前馈层替换为逐位置专家混合（MoE）层 [16]，并采用一种 top-2 门控的变体（图3）。我们通过调整 Transformer 层数以及每个 MoE 层的专家数量来缩放模型容量。

每个训练样本由一对子词标记序列组成。每个标记在训练和推理过程中都会激活 MoE Transformer（混合专家 Transformer）的一个子网络。该子网络的规模与每个 MoE 层的专家数量大致无关，从而使计算成本如上一节所述实现次线性扩展。第 3.1 节将进一步分析计算复杂度，第 5 节讨论训练性能。

2.2 逐位置专家混合层

我们所用的专家混合（MoE）层基于 [16]，并在所使用的稀疏门控函数和辅助损失上做了改动。用于 Transformer 架构的 MoE 层由 E 个前馈网络 前馈网络₁ ... 前馈网络 _{E} 组成：

$$\mathcal{G}_{s,E} = \text{GATE}(x_s) \quad (1)$$

$$\text{FFN}_e(x_s) = w_{oe} \cdot \text{ReLU}(w_{ie} \cdot x_s) \quad (2)$$

$$y_s = \sum_{e=1}^E \mathcal{G}_{s,e} \cdot \text{FFN}_e(x_s) \quad (3)$$

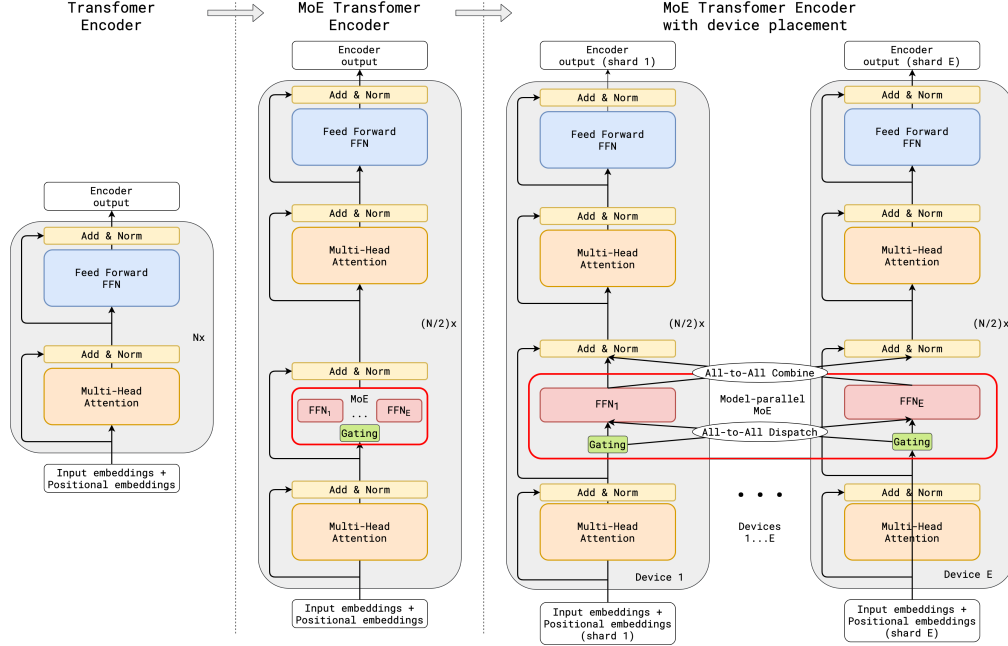


图3: 带有 MoE 层的 Transformer 架构编码器的扩展性示意图。MoE 层替换每隔一个的 Transformer 架构前馈层。解码器的修改类似。(a) 标准的 Transformer 模型的编码器是由自注意力和前馈层交替堆叠而成, 并穿插残差连接和层归一化。(b) 通过将每隔一个前馈层替换为 MoE 层, 即得到 MoE Transformer (混合专家 Transformer) 编码器的模型结构。(c) 当扩展到多个设备时, MoE 层进行跨设备分片, 而所有其他层则复制。

其中, x_s 是 MoE 层的输入标记, w_i 和 w_o 分别是前馈层 (一个专家) 的输入和输出投影矩阵。向量 $\mathcal{G}_{s,E}$ 由门控网络计算。 $\mathcal{G}_{s,E}$ 对每个专家都有一个非负值, 其中大多数为零, 表示该标记不会被分发到该专家。该标记仅被分发到极少数专家。我们选择让每个标记最多分发到两个专家。在 $\mathcal{G}_{s,E}$ 中对应的条目为非零, 表示某个专家对最终网络输出的贡献程度。每个专家前馈网络 (FFN) e 对 x_s 应用一个使用 ReLU [29] 激活函数的全连接两层网络。MoE 层的输出, y_s , 是所有被选中专家输出的加权平均。

门控函数 $\text{GATE}(\cdot)$ 对 MoE 层至关重要, 其采用 Softmax 激活函数建模, 用于指示各专家在处理入站令牌时的权重。换言之, 用于指示某个专家处理该入站令牌的能力高低。此外, 门控函数必须满足两个目标:

- **负载均衡** 理想情况下, MoE 层应对给定的标记稀疏地激活专家。一个朴素的方案是根据 Softmax 概率分布仅选择前 k 个专家。然而, 众所周知, 这种做法在训练 [16] 时会导致负载不均衡问题: 训练期间看到的大多数 Token 会被分发给少数几个专家, 导致只有少数 (繁忙的) 专家积累了非常大的输入缓冲区, 使得其他专家得不到充分训练, 从而减慢训练速度。与此同时, 许多其他专家几乎得不到充分训练。更好的门控函数设计会将处理负载更均匀地分配到所有专家上。
- **规模化效率** 如果以顺序方式执行门控函数, 实现负载均衡将相当简单。仅门控函数的计算成本, 对于给定 E 个专家、输入批次中所有 N 个标记, 至少为 $O(NE)$ 。然而, 在我们的研究中, N 的数量级为百万, E 的数量级为千, 顺序实现的门控函数将使大部分计算资源在大多数时间处于空闲状态。因此, 我们需要对门控函数进行高效的并行实现, 以利用众多设备。

我们在门控函数 $\text{GATE}(\cdot)$ 中设计了以下机制，以满足上述需求（细节如算法 1 所示）：

- **专家容量** 为确保负载均衡，我们强制每个专家处理的 Token 数量低于某个统一阈值，我们将其定义为专家容量。假设一个训练批次中的 Token 总数为 N ，且每个标记最多被分发到两个专家，则将专家容量设为 $O(N/E)$ 。 $\text{GATE}(\cdot)$ 维护一个累计计数器 c_e 用于统计分发到某个专家的 Token 数量。当一个标记选择的两个专家都已超过其容量时，该标记被视为溢出标记，其中 $\mathcal{G}_{s,E}$ 退化为零向量。此类标记将其表示 x_s 通过残差连接传递到下一层。
- **本地分组分发** $\text{GATE}(\cdot)$ 将一个训练批次中的所有 Token 平均划分为 G 个分组，即每个分组包含 $S = N/G$ 个 Token。所有分组都独立并行处理。每个分组被分配每个专家的容量份额，即 $2N/(G \cdot E)$ 。每个分组确保分发到某个专家的标记数量至多为该数目。这样即可确保仍然遵守专家容量，且整体负载均衡。
- **辅助损失** 关键在于门控函数不要总是选择同样的少量专家，否则会导致只有少量专家出现容量溢出，而其余专家未被充分利用。按照 [16]，我们定义一个辅助损失项 ℓ_{aux} 来施加这一约束。它以常数乘子 k 添加到模型 $\mathcal{L} = \ell_{nll} + k * \ell_{aux}$ 的整体损失函数中。算法 1 的第 (13) 行中辅助损失项 ℓ_{aux} 的具体形式源于如下考虑：项 c_e/S 表示已路由到每个专家的输入所占比例，我们希望最小化 c_e/S 的均方。但由于 c_e 源自 top-2 操作且不可微，我们使用每个专家的平均门控 m_e 作为可微近似，并将 $(c_e/S)^2$ 替换为 $m_e(c_e/S)$ ，从而可用梯度下降进行优化。
- **随机路由** 直观地说，由于 y_s 是所选专家返回结果的加权平均，如果第 2 个专家的权重非常小，我们可以直接忽略第 2 个专家，以节省整体的专家容量。因此，除遵守专家容量约束之外， $\text{GATE}(\cdot)$ 还会以与其权重 g_2 成比例的概率分发到次优专家。

3 使用 GShard 的高度并行实现

本节描述第 2 节中的模型在 TPU 设备集群上高效运行的实现。

第一步是用线性代数运算来表述该模型，我们的软件栈（TensorFlow [21]）和硬件平台（TPU）都针对这类运算进行了高度定制与优化。与原始的 Transformer 架构相同，可以很容易地用线性代数来实现模型的大部分部分。然而，由于其顺序性，要表述 MoE 层，尤其是算法 1 中给出的 $\text{GATE}(\cdot)$ 函数，需要投入一些额外工作；相关细节见第 3.1 节。

接下来，我们为线性代数计算添加注解以表达并行。可以使用第 3.2 节中的分片 API，将计算中的每个张量注解为在设备集群上复制或分布。使用分片注解能够在模型描述与高效并行实现之间实现关注点分离，并允许用户灵活表达多样的并行化策略。例如，(1) 注意层通过沿批次维度切分并行，并将其权重复制到所有设备。另一方面，(2) 由于规模巨大，MoE 层中的专家无法在所有设备上复制，唯一可行的策略是将专家分片到多个设备上。此外，整个模型在这两种模式 (1)-(2) 之间交替。使用注解使模型开发者免于系统优化工作，并避免将并行实现和底层细节固化到模型代码中。

最后，编译器基础设施把一个（部分）带注释的线性代数计算转化为可缩放到数千台设备的高效并行程序。正如将在第 3.3 节中所述，编译器应用 SPMD(Single Program MultipleData) 划分变换来表达每个设备上的计算，插入必要的跨设备通信，并处理不规则的

算法 1: 带有辅助损失的组级 top-2 门控

数据: x_S , 大小为 S 的一组 Token
数据: C , 分配给该组的专家容量
结果: $\mathcal{G}_{S,E}$, 分组组合权重
结果: ℓ_{aux} , 分组辅助损失

```
(1)  $c_E \leftarrow 0$                                 ▷ 每个专家的门控决策
(2)  $g_{S,E} \leftarrow \text{softmax}(wg \cdot x_S)$         ▷ 每个标记对每个专家的门控值,  $wg$  是可训练的权重
                                                ▷ 每个专家的门控均值
(3)  $m_E \leftarrow \frac{1}{S} \sum_{s=1}^S g_{s,E}$ 
(4) for  $s \leftarrow 1$  to  $S$  do
(5)    $g1, e1, g2, e2 = \text{top\_2}(g_{s,E})$           ▷ top-2 门控和专家索引
(6)    $g1 \leftarrow g1 / (g1 + g2)$                 ▷ 归一化  $g1$ 
(7)    $c \leftarrow c_{e1}$                             ▷ 在  $e1$  专家缓冲区中的位置
(8)   如果  $c_{e1} < C$  则
(9)      $\mathcal{G}_{s,e1} \leftarrow g1$                     ▷  $e1$  专家组合权重用于  $x_s$ 
(10)  end
(11)   $c_{e1} \leftarrow c + 1$                         ▷ 递增  $e1$  专家决策计数
(12) 结束
(13)  $\ell_{aux} = \frac{1}{E} \sum_{e=1}^E \frac{c_e}{S} \cdot m_e$ 
(14) 对于  $s \leftarrow 1$  到  $S$  执行
(15)    $g1, e1, g2, e2 = \text{top\_2}(g_{s,E})$           ▷ top-2 门控和专家索引
(16)    $g2 \leftarrow g2 / (g1 + g2)$                 ▷ 归一化  $g2$ 
(17)    $rnd \leftarrow \text{uniform}(0, 1)$               ▷ 以  $\propto 2 \cdot g2$  的概率分发到次优专家
(18)    $c \leftarrow c_{e2}$                             ▷  $e2$  专家缓冲区中的位置
(19)   如果  $c < C \wedge 2 \cdot g2 > rnd$  则
(20)      $\mathcal{G}_{s,e2} \leftarrow g2$                     ▷  $e2$  针对  $x_s$  的专家组合权重
(21)   end
(22)    $c_{e2} \leftarrow c + 1$ 
(23) 结束
```

模式, 例如不均匀划分, 并最终生成一个单一程序, 在所有设备上启动以进行并行执行。

3.1 用线性代数表示的逐位置专家混合层

我们的模型实现 (算法2) 将整个加速器集群视为单个设备, 并用与集群具体配置无关的少量张量运算表达其核心数学算法。爱因斯坦求和记号 [30] (即 `tf.einsum`) 是一种能够简洁表达模型的强大构造, 我们在实现中大量使用。Softmax 门控的计算可以由一次 `einsum` 后接 Softmax 函数来直接表示。将输入分发到选定专家可由分发掩码与输入之间的一次 `einsum` 表达。所有前馈网络 e 权重被合并为单个三维张量 wi 和 wo , 而由前馈网络 $1 \dots$ 前馈网络 E 的计算使用 3 个算子表示 (两次 `einsum` 和一次 ReLU)。最后, 将所有专家的输出按权重求平均得到最终输出, 可用另一条 `einsum` 表达。

算法2中的 Top2 门控计算算法 1 中所描述的所有组内 $\mathcal{G}_{S,E}$ 的并集。combine_weights 是一个形状为 $[G, S, E, C]$ 的四维张量。当组 g 中的输入标记 s 被发送到专家 e 的输入缓冲区的缓冲区位置 c 时, 值 `combine_weights[g, s, e, c]` 非零。对于特定的 g 和 s , 切片 `combine_weight[g, s, :, :]` 至多包含两个非零值。二值 `dispatch_掩码` 由 `combine_weights` 生成, 只需将所有非零值设为 1。

我们需要适当地选择组数 G 和专家数量 E , 以便该算法可以缩放到具有 D 设备的集群。给定一个包含 N Token 的训练批次, 分析其在一次训练步数中的总体计算复杂度 (浮点运算总数) 是值得的。

算法2: 按位置的专家混合层的前向传播。带下划线的字母（例如，**G** 和 **E**）表示张量将沿其进行划分的维度。

```
1 门控 = Softmax ( einsum ("GSM ,ME ->GSE ", inputs , wg ))
2 合并_权重, 分发_掩码 = Top2Gating ( 门控 )
3 已分发的_专家_输入 = einsum ( 4 "GSEC ,GSM ->EGCM ", 分发_掩码, 重塑后的_输入 )
5 h = einsum ("EGCM ,EMH ->EGCH ", 已分发的_专家_输入, wi )
6 h = relu (h)
7 专家_输出 = einsum ("EGCH ,EHM ->GECM ", h , wo )
8 输出 = einsum ( 9 "GSEC ,GECM ->GSM ", 合并_权重, 专家_输出 )
```

我们分析算法2的计算复杂度随设备 D 数量的扩展性，基于以下假设： a) 每个设备的令牌数量 $\frac{N}{D} = O(1)$ 为常数¹； b) $G = O(D)$ 、 $S = O(1)$ 和 $N = O(GS) = O(D)$ ； c) $M = O(1)$ 、 $H = O(1)$ ； d) $E = O(D)$ ；以及 e) $C = O(\frac{2S}{E}) = O(\frac{1}{D})$ $D < S$ 并且是一个正整数²。

算法2 中的浮点运算总次数 $FLOPS$ ：

$$\begin{aligned} & FLOPS_{\text{Softmax}} + FLOPS_{\text{Top2Gating}} + FLOPS_{\text{Dispatch/Combine}} + FLOPS_{\text{FFN}} = \\ & O(GSME) + O(GSEC) + O(GSMEC) + O(EGCHM) = \\ & O(D \cdot 1 \cdot 1 \cdot D) + O(D \cdot 1 \cdot D \cdot \frac{1}{D}) + O(D \cdot 1 \cdot 1 \cdot D \cdot \frac{1}{D}) + O(D \cdot D \cdot \frac{1}{D} \cdot 1 \cdot 1) = \\ & O(D^2) + O(D) + O(D) + O(D) \end{aligned}$$

因此，每个设备的 $FLOPS/D = O(D) + O(1) + O(1) + O(1)$ 。每个设备的 Softmax 复杂度 $FLOPS_{\text{softmax}}/D = O(D)$ 随设备数量线性增长，但在实践中由于 $D \ll H$ 和 $D < S$ ，它被其他项所主导。因此， $FLOPS/D$ 可以被视为 $O(1)$ ，满足次线性扩展的设计要求。第 5 节通过实证验证了这一分析。

除了计算开销之外，我们还有非常数的跨设备通信开销，但当我们增大 D （第 5 节）时，它以适度的速率 $O(\sqrt{D})$ 增长。

3.2 GShard 注解 API 用于并行执行

由于算法 1 中张量的规模和计算需求极为庞大，我们必须在许多设备上将该算法并行化。关于如何对算法2 中的每个张量进行分片，一个直接的方案由算法2 中带下划线的字母所示。

GShard 中的 分片 API 允许我们对程序中的张量进行注解，从而有选择地指定它们应如何被划分。这些信息会被传播到编译器，以便编译器能够自动应用转换以实现并行执行。我们在工作的过程中使用 TensorFlow/Lingvo [31] 中的以下 API。

- **replicate(tensor)** 将张量标注为跨分区复制，并返回该已标注的张量。这通常用于我们模型中的非MoE层以复制权重。
- **split(tensor, split_dimension, num_partitions)** 将张量标注为沿着 `split_dimension` 进行分区，并返回该已标注的张量。分区 i 被放置在第 i 个设备上，并且 `num_partitions` 不得超过系统上的设备数量。
- **shard(tensor, device_assignment)** 将 `split()` 泛化，使其允许对多个维度进行划分，并指定每个分区的放置位置。附录A.3 对该 API 进行了更详细的说明。

¹在实践中，这通常是为了避免设备内存溢出所必需的。²扩展性 $D > S$ 将需要以不同方式使用非整数专家容量。

请注意，对 `split` 或 `shard` 的调用仅添加注解，不会改变用户程序中的逻辑形状。用户仍然使用完整的形状，无需担心诸如不均匀划分之类的问题。

GShard 的通用性在于，这些简单的 API 以相同方式适用于所有维度。根据用例，分片的维度可以包括批次（数据并行）、特征、专家，甚至图像模型中的空间维度。此外，由于分片注解是针对每个张量的，模型的不同部分可以采用不同的划分方式。这种灵活性使我们能够对巨大的专家混合权重进行划分，并在 MoE 层与非 MoE 层之间切换分区模式，同时也适用于超出本论文范围的应用例，例如对大图像进行空间分区 [32] (附录A.4)。

借助上述分片 API，我们可以如下表示 算法2 所示的分片策略。输入张量沿第一维度进行 `split`，并将门控权重张量复制。计算出分发到各专家的输入后，我们应用 `split` 将分片从组 (G) 维度切换到专家 (E) 维度。 D 是设备数量。

```
1 # 沿组 (G) 维度划分输入 .2 +inputs =split ( inputs , 0, D) 3 # 复制 门控 权重4 +
wg =replicate ( wg ) 5 gates =softmax ( einsum ( " GSM ,ME ->GSE " , inputs , wg )) 6
combine_weights , dispatch_mask =Top2Gating ( gating_logits ) 7 dispatched_expert_
inputs = einsum ( 8 " GSEC , GSM ->EGCM " , dispatch_mask , reshaped_inputs ) 9 # 沿 专
家 (E) 维度划分 已分发 输入 .10 +dispatched_expert_inputs =split ( dispatched_expert_
inputs , 0, D) 11 h =einsum ( " EGCM , EMH ->EGCH " , dispatched_expert_inputs , wi ) 12 ...
```

逐张量分片分配 如上例所示，用户无需为程序中的每个张量添加注解。注解通常只需加在少数重要算子上（例如我们模型中的 `Einsum`），其余张量的分片由编译器使用自身的启发式方法进行推断³。例如，由于输入张量沿 G 划分，且权重张量被复制，编译器会将该 `einsum` 的输出沿相同的 G 维度划分（第 5 行）。类似地，对于用于输入分发的 `einsum`（第 7 行），由于两个输入都沿 G 维度被划分，因此推断输出分片将沿 G 维度进行切分，随后我们在输出上添加 `split` 注解，以沿 E 维度重新分片。上述示例中的某些注解也可由编译器确定（例如，`replicate(wg)`），但建议对计算的初始输入张量和最终输出张量进行注解。

编译器目前使用迭代的数据流分析，自用户标注的算子开始，将分片信息从一个算子传播到其邻居（操作数和使用者）。该分析通过对齐相邻算子的分片决策，尽量减少重新分片的可能性。也可以采用其他方法，例如整数规划或机器学习方法，但改进自动分片分配并非本论文的重点，我们将其留作未来工作。

混合手动与自动分片 使用分片注解的自动分片在常见场景下通常已足够，但 GShard 也具备灵活性，允许将手动分片的算子与自动分片的算子混合使用。这为用户提供了更多对算子如何分片的控制；例如，用户可能掌握超出算子语义之外的运行时知识。例如，XLA 和 TensorFlow 的 `Gather` 算子定义都不会传达关于输入中不同范围的索引边界的信息，但用户可能知道某个特定的 `Gather` 算子仅在每个分区内重排数据。在这种情况下，用户只需缩小维度大小并执行本地 `Gather`，便可轻松对该算子进行分片；否则，编译器就不得不对索引范围采取保守策略，从而引入不必要的通信开销。例如，算法2中的用于分发的 `Einsum`（第 3 行）

³对于编译器来说，推断缺失的分片也很重要，因为反向传播计算通常由前端框架自动生成，用户无法访问这些张量。

在算法2中，使用 one-hot 矩阵来分发输入，也可以改由采用简单手动分片的 Gather 算子来实现，而模型的其余部分由自动分片完成。下面是演示该用例的伪代码。

```
1 # 输入的形狀为 [G, S (序列长度), M] .split () 不会改变逻辑形状 .2 input = split (
input, 0, num_devices) 3 # s_indices 的形狀为 [E (本地专家数), G, C (专家容量)], 1 值:
输入中 S (序列长度) 的索引 .4 s_indices = split (s_indices, 1, num_devices) 5 6 # 开始手动分
区 .7 # partitioned_input 的形狀为 [G/num_devices, S (序列长度), M] 8 partitioned_input =
auto_to_manual_spmd_partition (input) 9 # partitioned_s_indices 的形狀为 [E (本地专家数)
, G/num_devices, C (专家容量)] 10 partitioned_s_indices = auto_to_manual_spmd
partition (s_indices) 11 # 使用 partitioned_input 中的 G 索引进行 concat. 在 G 维度上执行
iota 12 partitioned_s_indices = concat (iota (E, G/num_devices, C), 1, 0)
partitioned_s_indices, 5) 14 # partitioned_data 的形狀为 [E (本地专家数), G/num_devices,
C (专家容量), M] 15 partitioned_data = gather (16 partitioned_input, partitioned_gs_
indices) 17 18 # 切换回自动分区 .19 # 数据的形狀为 [E (本地专家数), G, C (专家容量), M]
20 data = manual_to_auto_spmd_partition (partitioned_data) 21 ...
```

3.3 用于 GShard 的 XLA SPMD 分区器

本节介绍一种编译器基础设施，它基于分片注解自动对计算图进行分区。分片注解告知编译器每个张量应如何跨设备分布。SPMD（单程序多数据）分区器（为简便起见，简称“分区器”）是一个编译器组件，它将计算图转换为一个在所有设备上并行执行的单一程序。这使得无论分区数量多少，编译时间几乎保持不变，从而使我们能够缩放到数千个分区。⁴

我们在 XLA 编译器 [28] 中实现了该分区器。包括 TensorFlow、JAX、PyTorch 和 Julia 在内的多个前端框架已经具备 lowering 逻辑，可将其图表示转换为 XLA HLO 图。与 TensorFlow 等流行前端框架相比，XLA 的算子集合也要小得多，这在不损害通用性的情况下降低了实现分区器的负担，因为前端现有的 lowering 已经完成了使其具有表达力的大部分繁重工作。尽管我们在 XLA 中开发了这套基础设施，我们在此描述的技术也可应用于其他机器学习框架的中间表示（例如，ONNX [33], TVM Relay [34], Glow IR [35]）。

XLA 将一次计算建模为一个数据流图，其中节点是算子，边是流动于算子之间的张量。分片器的核心是逐算子处理：根据输入和输出上指定的分片，将完整尺寸的算子转换为分片尺寸的算子。对一个计算进行分片时，会引入多种跨设备数据传输模式。为了在大规模下最大化性能，关键是定义一组核心通信原语，并针对目标平台对其进行优化。

3.3.1 通信原语

由于分区器强制所有设备运行相同的程序，通信模式也很规整，XLA 定义了一组用于执行 MPI 风格通信 [36] 的集体算子。下面我们列出在 SPMD 分区器中使用的常见通信原语。

⁴另一种选择是 MPMD（Multiple Program Multiple Data），其无法缩放，如图2所示。

CollectivePermute 该算子指定一组源-目的对，源的输入数据会被发送到相应的目的端。它有两个用法：在各分区之间改变分片张量的设备顺序，以及本节稍后将讨论的 **halo exchange**。

AllGather（全收集） 该算子按照指定顺序将所有参与者的张量拼接。它用于将分片张量转换为复制张量。

AllReduce 该算子对来自所有参与者的输入执行逐元素规约（例如求和）。它用于合并来自不同分区的部分规约的中间表示张量。在 TPU 设备网络中，当分区数量增加时，**AllReduce** 的开销保持不变（第 5.2 节）。它也是一种常用的原语，并在其他类型的网络拓扑中具有高效的实现 [37]。

AllToAll（全互连） 该算子在逻辑上沿某一维度将每个参与者的输入拆分，然后将每一块发送给不同的参与者。收到来自其他参与者的数据块后，每个参与者将这些块拼接以生成其结果。它用于将分片张量从一个维度重新分片到另一个维度。在 TPU 设备网络中，**AllToAll（全互连）** 是执行此类重新分片的高效方式，其开销随着分区数量的增长呈次线性增加（第 5.2 节）。

3.3.2 逐算子的 SPMD 划分

划分器的核心是逐算子的变换：根据指定的分片，将完整规模的算子转化为分区规模的算子。虽然某些算子（例如，逐元素）支持起来很简单，但我们会讨论若干需要跨分区通信的常见情形。

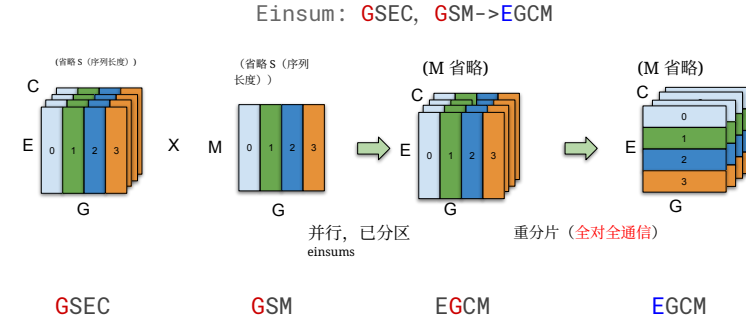
在一般情况下存在一些重要的技术挑战，我们将在第 3.3.3 节中讨论。为使讨论更贴近专家混合模型，本节聚焦 **Einsum** 的划分，以说明几种通信模式。此外，为了简化起见，我们假设所有张量都被均匀划分，即要划分的维度大小是分区数的整数倍。

Einsum 案例研究 **Einsum** 是实现专家混合模型中最关键的算子。它们在 XLA HLO 中表示为 **Dot** 操作，其中每个操作数（LHS 或 RHS）由三类维度组成：

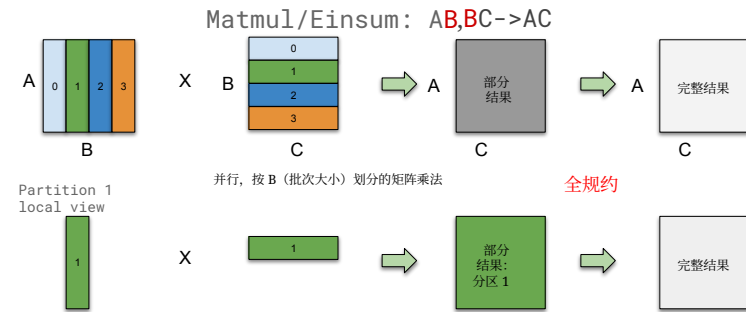
- **批次维度** 是天然并行的维度。在 LHS、RHS 和输出中必须存在相同的一组批次维度，且输出中的每个元素仅依赖于 LHS 和 RHS 中对应的批次。
- **收缩维度** 仅存在于操作数中。LHS 和 RHS 必须具有相同的一组收缩维度，并且这些维度在输出中被求和并折叠。
- **非收缩维度** 也是存在于某个操作数以及输出中的并行维度。LHS 和 RHS 各自有自己的一组非收缩维度，这些维度会被输出继承。

分片传播会优先在 LHS、RHS 和输出的批次维度上选择相同的分片，因为这样可以避免任何跨分区通信。然而，这并非总是可行，在以下三种情况下我们需要进行跨分区通信。

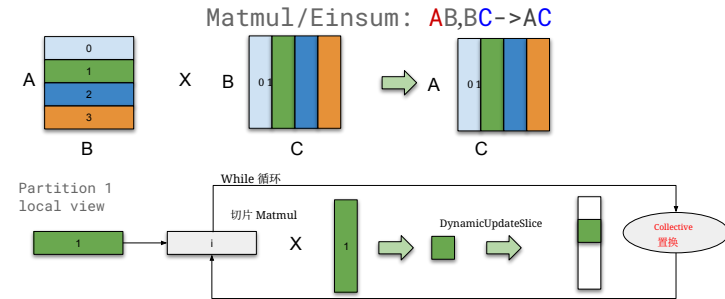
- **重新分片**。在我们构建的专家混合模型中，专家分发逻辑（算法 2 的第 3 行）要求在一次 **Einsum** 之后切换被分区的维度。由于通过 **AllToAll（全互连）** 进行重新分片在第 5.2 节中被证明是高效的，我们先在本地执行该 **Einsum**，然后将其重新分片到所需的维度，如图 4a 所示。
- **累加部分结果**。如果输入沿着收缩维度进行划分，则本地结果仅为部分结果，我们需要使用 **AllReduce** 将它们合并以得到最终结果，如图 4b 所示。
- **在循环中切片**。对于某些场景，我们还实现了一种类似于 **Cannon's algorithm** 的算法 [38]，以限制每个分区上的张量大小。例如，



(a) 一个被划分的 Einsum 算子。带颜色的字母 (G 和 E) 表示每个张量被划分的维度。划分器决定先沿着 G 维度执行批次并行的 Einsum, 然后将结果重新分片到 E 维度。



(b) 一个在收缩维度上划分的简单 Einsum (Matmul)。



(c) 一个 Einsum (Matmul), 我们在循环中使用 collective-permute 来计算每次计算一个切片。整个过程中不会出现完整大小的张量。

图4: 带有跨设备通信的 Einsum 划分示例。

如果两个操作数都在非收缩维度上被分区, 我们无法直接计算本地 Einsum, 因为操作数的非收缩维度不同。复制其中一个操作数不会导致冗余计算, 但要求被复制的操作数能够放入设备内存。因此, 如果操作数的大小过大, 我们会保持两个操作数都处于分区状态, 使用循环遍历结果的每个切片, 并使用 CollectivePermute 通信输入切片 (图 4c)。

3.3.3 支持完成的算子集合

我们还解决了若干额外的挑战, 使 SPMD 分区器能够在不对张量形状或算子配置施加额外约束的情况下, 支持完整的算子集。这些挑战常常涉及分区之间不对称的计算或通信模式, 这在

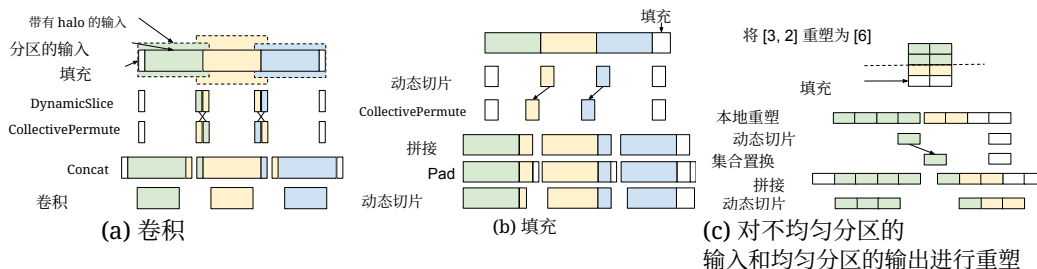


图5: Halo 交换示例。

SPMD 中难以表达，因为单一程序需要对所有分区都足够通用。我们不能仅仅基于运行时的设备ID在单一程序中创建许多分支，因为那会导致程序规模爆炸。

静态形状与非均匀分区 XLA 要求张量形状为静态。⁵ 然而，当一次计算被分区时，并非所有分区都一定具有相同的输入/输出形状，因为维度可能无法被分区数整除。在这种情况下，形状的大小会向上取整到分区数的下一个倍数，且该填充区域中的数据可以是任意的。

在计算某个算子时，为保证正确性，我们可能需要向填充区域填入已知数值。例如，如果需要对 **Reduce-Add** 算子进行分区，则需要使用恒等函数的零值。考虑一个示例：被分区的维度（15）不能被 2（分区数）整除，因此分区 1 比所需多出一列。我们创建一个范围为 [0, 8) 的 **Iota** 算子，加上分区偏移量（由 $PartitionId \times 8$ 计算得到），并与完整形状的偏移量（15）进行比较。根据谓词的数值，从操作数或从零中进行选择，结果就是掩码后的操作数。

静态算子配置 XLA 算子具有静态配置，例如在 **Convolution** 中定义的填充、步幅和膨胀。然而，不同分区可能不会以相同的算子配置执行。例如，对于 **Convolution**，最左侧分区在其左侧应用填充，而最右侧分区在其右侧应用填充。在这种情况下，分区器可能会选择一种配置，使某些分区生成比所需略多的数据，然后切掉无关部分。附录A.4 讨论了 **Convolution** 及类似算子的示例。

Halo 交换 某些算子具有一种通信模式，涉及与相邻分区进行部分数据交换，我们称之为 **Halo 交换**。我们使用 **CollectivePermute** 算子在分区之间交换 halo 数据。

Halo 交换最典型的用例是对基于窗口的算子进行分区（例如，**Convolution**、**ReduceWindow**），因为相邻分区可能需要重叠的输入数据（图5a）。在实践中，这些算子的 Halo 交换通常需要与适当的填充、切片和掩码配合使用，原因在于复杂的窗口配置（膨胀、步幅和填充）的使用，以及不均匀的 halo 大小。我们在附录A.4 中描述了各种情形。

halo 交换的另一个用途是用于那些会改变形状大小的数据格式化算子。例如，在经过一个 **Slice** 或 **Pad** 算子之后，张量的形状会发生变化，分区之间的边界也会随之改变。这要求我们在不同分区上重新对齐数据，可以将其视作一种 halo 交换来处理（图5b）。

其他数据格式化算子，尽管在逻辑上不改变形状的大小，也可能需要进行 halo 交换，尤其是由于静态形状约束和不均匀划分。比如，**Reverse** 算子会反转张量中元素的顺序，但如果其被不均匀划分，我们需要在分区之间移动数据，以使填充在逻辑上位于结果张量的右侧。另一个例子是 **Reshape**。考虑将一个张量从 [3, 2] 变形为 [6]，其中输入

⁵在中间表示中的有限动态性常常是高效地面向

加速器所必需的。

在第一维被以 2 路不均匀划分（划分形状为 [2, 2]），并且输出也以 2 路划分（划分形状为 [3]）。由于不均匀划分，输入上存在填充，但在 Reshape 之后，输出张量不再有填充；因此，需要类似于 Slice 的 halo 交换（图5c）。

编译器优化 SPMD 分区器会创建各种数据格式化算子，以执行切片、填充、拼接、掩码和 halo 交换。为了解决这一问题，我们在 TPU 上利用 XLA 的融合能力，并对切片和填充应用代码移动优化，从而在很大程度上隐藏数据格式化的开销。其结果是，运行时开销通常可以忽略不计，即使对于大量使用掩码和填充的卷积网络也是如此。

4 大规模多语言、大规模机器翻译（M4）

4.1 多语言翻译

我们选择多语言神经机器翻译（MT）[39, 40, 41] 来验证我们使用 GShard 进行高效训练的设计。多语言 MT 本质上是一个多任务学习问题，其目标是构建一个单一神经网络，以同时翻译多个语言对。这将我们的研究路线拓展 [15, 14, 16] 到通用机器翻译模型 [42]，即一个能够在所有领域中在一百多种语言之间进行翻译的单一模型。此类大规模多语言翻译模型不仅便于在大规模下对模型进行压力测试，而且已被证明在真实世界的生产系统中具有实际影响 [43]。

在极大规模多语言的 MT 中，就模型质量而言，衡量成功的标准有两点：1) 在拥有大量训练数据的语言（高资源）上的提升；2) 在数据有限的语言（低资源）上的提升。随着单个翻译模型中需建模的语言对（任务）数量增加，正向语言迁移 [44] 开始为低资源语言带来显著收益。鉴于所涉及的语言数量，M4 在提升低资源任务方面具有明显优势。相反，对于高资源语言，任务数量的增加会限制模型中每个任务的容量，导致其翻译质量低于只在单一语言对上训练的模型。这种面向高资源语言的容量瓶颈可以通过将模型规模扩展到极大规模来缓解，以满足额外容量的需求 [14, 15]。

因此，极大规模多语言、超大规模的 MT 旨在通过极大规模多语言性提升正向迁移 与通过大规模扩展缓解容量瓶颈 之间取得平衡。同时，模型规模与所涉及语言数量的扩展必须配合合适的神经网络架构。为放大正向迁移 并降低负迁移⁶，一种自然的做法是设计一种模型架构：在不同语言间容纳共享组件（共享子网络），同时保留一些特定于语言的组件（非共享、语言特定的子网络）。然而，随着语言数量的增加，模型设计中的搜索空间（决定共享什么）迅速膨胀，使得基于启发式的方法去搜索合适的架构变得不切实际。因此，基于从数据中学习神经网络连线模式的方法应运而生，成为具备可扩展性且切实可行的前进方向。

在本节中，我们阐述条件计算 [45, 46] 结合稀疏门控的专家混合 [16] 如何契合上述详述的期望特性，并通过**将神经机器翻译模型扩展到超过 1 万亿参数**来展示其有效性，同时使如此庞大的网络的训练时间保持可行。例如，面向 M4 的 600B GShard 模型可处理 1T 个标记⁷ 在 250k 个训练步数内，并在不到 4 天内完成。我们通过向模型中不断添加更多专家来提升模型容量，并研究在收敛、模型质量和训练效率中起作用的因素。此外，我们展示了条件计算如何加速训练 [25] 以及如何在无需任何关于任务或语言相关性的先验知识的情况下，高效地学习在网络中对每个标记进行稀疏门控/路由，体现了可直接从数据中学习路由决策的能力。

⁶负迁移是指不相关的任务共享模型容量，这反过来会损害此类相互干扰的任务的质量。⁷子词分割后的源端标记。

4.2 数据集与基线方法

逐步扩大模型以获得更高质量的前提是，一开始就需要大量的训练数据 [3]。遵循此前关于稠密扩展性的多语言机器翻译 [15, 14], 相关工作，我们选择在 MT 在真实环境中这一贴近现实的测试平台上开展研究，并使用一个 Web 规模的自建数据集。从网络挖掘得到的训练语料 [47], 包含 100 种语言与英语之间的平行文档，总计 250 亿个训练样本。该训练集有几项特征值得一提。由于来自网络，联合语料噪声相当大，同时覆盖了多样的领域和语言。如此广泛的覆盖也带来了语言间在每个语对样本数量方面的严重不均衡。这种不均衡遵循陡峭的幂律分布：高资源语言拥有数十亿个样本，而低资源语言只有数万个。上述特性虽然给我们的研究带来挑战，但也使整体尝试尽可能贴近真实。有关所用数据集的更多细节，请参见 [15, 14]。

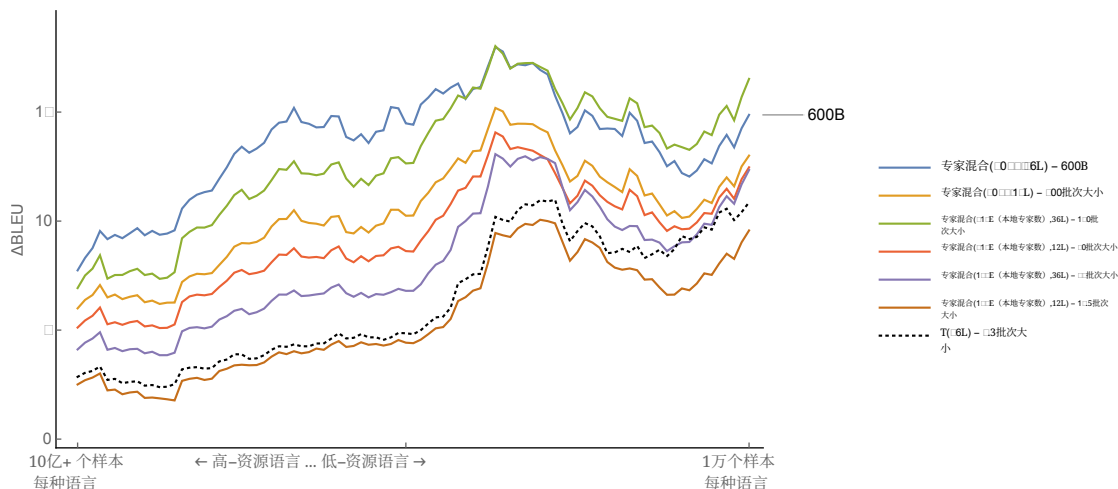
我们专注于提升从所有 100 种语言到英语的翻译质量（以 BLEU 分数 [48] 衡量）。这带来了约 130 亿个用于模型训练的样本⁸。为了构建我们的基线方法，我们为每个语言对训练了单独的双语神经机器翻译模型（例如用于德语到英语的单个模型），并根据每种语言可用的训练数据进行调参⁹。我们不展示每个语言对的单独 BLEU 分数，而是遵循一种惯例：将基线方法在 x 轴上置于零处，并报告使用 GShard 训练的每个大规模多语言模型的 Δ BLEU 趋势线（见图 6）。图 6 中的 x 轴按可用训练数据量从左到右递减排序，最左侧对应高资源语言，最右侧对应低资源语言。重申一下，我们在通用机器翻译中的最终目标是使单个多语言模型的 Δ BLEU 趋势线在所考虑的所有语言上都超越基线方法。我们还将使用 GPipe 管道并行在相同数据集上训练的稠密 96 层 Transformer 架构编码器-解码器网络 T(96L) 的一个变体作为另一条基线（图 6 中的虚线趋势线）。在 2048 个 TPU v3 核心¹⁰上训练至收敛耗时超过 6 周，性能超越原始 GPipe T(128L)¹¹ [15]，并且是我们比较中使用的最强单个稠密模型基线。

4.3 稀疏门控 专家混合 Transformer 架构：模型与训练

对 Transformer 架构的扩展性近来一直是一条探索性的研究方向 [49, 50, 51]。不失一般性地，新近的方法通常通过堆叠越来越多的层来扩展 Transformer [49, 15]、拓宽网络的主导维度（即模型维度、隐藏维度或注意力头数）[4, 11]，以及通过架构搜索来学习连线结构 [52]¹²。对于大规模多语言机器翻译，[15] 展示了使用 GPipe pipeline parallelism 进行扩展性的最佳实践；其中，一个具有 128 层、60 亿参数的 Transformer 模型被证明能有效提升高资源语言，同时表现出最高的正向迁移 面向低资源语言。尽管前景非常可观，并且满足我们对通用翻译的期望特性，但对 Transformer 架构进行稠密扩展在实践中存在局限性，我们已在第 1 节的训练效率中提及。

我们的目标是实现可行的训练时间，并寻求能够保证训练效率的架构。我们的策略有三大支柱：通过堆叠更多层来增加网络的深度，类似于 GPipe [15]，增加网络的宽度，方法是引入多个前馈网络（专家）的副本，如第 2.2 节所述；并使用学习到的路由模块将 Token（以稀疏方式）分配给专家，如第 2.1 节所述。通过这三个组成部分，我们得到一种

⁸与使用相同数据集的既有工作相比，Kazakh 和 Latin 到 English 的语言对被排除在评估之外。⁹我们在 Transformer-Big 或 Transformer-Base 布局中分别针对高资源或低资源语言，调节了批大小以及不同的正则化方法取值（例如 Dropout（随机失活））。¹⁰T(96L) 在 300k 步数时处理约 1+ 万亿 Token，约 4M Token/步，总计 235.5 个 TPU v3 核心年¹¹64 个编码器 + 64 解码器层、16384 隐藏维度、32 个注意力¹²由于利用架构搜索的方法计算开销很大，因此不在本文的范围之内考虑。



Id	模型	BLEU avg.	ΔBLEU avg.	权重
(1)	专家混合(2048E, 36L)	44.3	13.5	600B
(2)	专家混合(2048E, 12L)	41.3	10.5	200B
(3)	专家混合(512E, 36L)	43.7	12.9	150B
(4)	专家混合(512E, 12L)	40.0	9.2	50B
(5)	专家混合(128E, 36L)	39.0	8.2	37B
(6)	专家混合(128E, 12L)	36.7	5.9	12.5B
*	T(96L)	36.9	6.1	2.3B
*	基线方法	30.8	-	100×0.4B

图 6: 使用 GShard 训练的多语言 MoE Transformer 模型与单语基线方法的翻译质量对比。沿着 x 轴的位置代表语言, 从高资源到低资源。 ΔBLEU 表示单个多语言模型相对于为特定语言训练并调优的单语 Transformer 模型的质量增益。使用 GShard 训练的 MoE Transformer 模型以实线趋势线表示。虚线趋势线表示使用 GPipe 在同一数据集上训练的单个 96 层多语言 Transformer 模型 T(96L)。为提高清晰度, 每条趋势线都使用长度为 10 的滑动窗口进行平滑。(彩色查看效果最佳)

easy 可缩放、训练高效且表达能力很强的架构, 我们称之为 Sparsely-Gate Mixture-of-Experts Transformer, 或简称 MoE Transformer (混合专家 Transformer)。

Model Details 为详细说明模型细节, 每个专家都被设计为与常规 Transformer 前馈网络相同的结构, 并且专家 (MoE 层) 以隔层分布于 Transformer 层中。为简化起见, 我们将训练使用的设备数量与每个 MoE 层的专家数量绑定, 尽管这并非必要要求。训练期间, 我们对模型权重和激活均使用 float32, 以确保训练稳定性。我们还在 MoE(2048E, 60L) 上进行了额外的可扩展性实验, 采用 bfloat16 [53] 激活, 总计 1 万亿个模型权重。尽管通过仔细的手动诊断可以进行训练, 但对于这个深层、具有 1 万亿权重的模型, 我们遇到了若干与数值稳定性相关的可训练性问题, 因此出于可复现性的考虑未包含这些结果。更多模型与训练细节请参见附录 A.2。

4.4 结果

在深入讨论训练效率的细节之前, 我们首先研究各种设计选择对构建 MoE Transformer (混合专家 Transformer) 的影响。为了剪枝搜索空间, 我们探索性地改变了两个

Id	模型	专家 每层	专家 总数	TPU v3 核心	编码器+解码器 层	权重
(1)	专家混合(2048E, 36L)	2048	36684	2048	36	600B
(2)	专家混合(2048E, 12L)	2048	12228	2048	12	200B
(3)	专家混合(512E, 36L)	512	9216	512	36	150B
(4)	专家混合(512E, 12L)	512	3072	512	12	50B
(5)	专家混合(128E, 36L)	128	2304	128	36	37B
(6)	专家混合(128E, 12L)	128	768	128	12	12.5B
*	专家混合(2048E, 60L)	2048	61440	2048	60	1T

表1: MoE Transformer (混合专家 Transformer) 模型家族。为了达到期望的容量, 我们*i)* 通过堆叠更多层来增加深度, *ii)* 通过将每个 MoE层的专家数量与用于训练的核心数量一起进行缩放来增加网络的宽度。

变量: Transformer 架构的编码器-解码器栈的层数 (记为 L) 以及每隔一层 MoE层 使用的专家总数 (记为 E (本地专家数))。对于深度, 我们测试了三种不同选项: 12 层 (原始的 Transformer 架构深度, 由 6 个编码器层和 6 个解码器层组成)、36 层和 60 层。对于替换每隔一层前馈层的专家数量, 我们同样测试了三种选项, 即 128、512 和 2048 个专家。请注意, 用于训练的设备数量固定为等于每层的专家数量, 分别使用 128、512 和 2048 个核心, 且与所实验的深度无关。有关模型配置的详细说明, 请参见表1。

对于每个实验 (表1的各行), 我们训练相应的 MoE Transformer (混合专家 Transformer) 模型, 直到其见过 1 万亿 (10^{12}) Token。我们使用此时的模型检查点进行模型评估。在这一点上, 我们在任何实验中都没有观察到过拟合模式。相反, 我们观察到如果继续更长时间的训练, 训练损失会持续改善。我们在一个保留测试集上评估了模型在所有语言对上的 BLEU 分数。图 6 报告了我们的全部结果。

在此, 我们针对每个实验分享定性分析, 并讨论每种设置对高资源和低资源语言的影响, 以跟踪我们朝通用翻译迈进的进展。为支撑接下来的分析, 值得重申我们对潜在质量提升的预期行为。为了在单个模型内同时提升高资源和低资源语言的质量, 规模化的模型必须通过为高资源任务分配足够的容量来缓解 *capacity bottleneck* 问题, 同时通过促进充分的参数共享来增强对低资源任务的 *positive transfer*。我们将此类系统的预期学习动态与由来已久的记忆与泛化两难松散关联起来, 近期已有沿着宽度 vs 深度的缩放努力开展的研究 [54]。我们不仅期望模型在保留测试集上具有更好的泛化, 还期望它们展现出跨语言的高迁移能力, 作为泛化性能的另一种体现 [55]。

更深的模型在各方面带来一致的质量提升 我们首先研究模型深度与模型质量之间的关系, 覆盖高资源与低资源语言。为在保持每层专家数量固定的同时测试泛化性能, 我们进行了三项不同的实验。随着每项实验中每层专家数量的增加 (128、512、2048), 我们在每种专家规模下将网络的深度提升三倍, 从 12 增至 36。这形成了三个组: 在每个组内, 每层专家数固定, 但深度提升为原来的三倍:

对于图 6 所示的每个配置, 我们观察到: 在保持每层专家数 (E) 不变的情况下增加深度 (L), 会在低资源和高资源语言上带来一致的质量提升 (沿着 y 轴向上 Δ 位移), 并且几乎在每次将深度从 12L 缩放到 36L 时都体现为近似恒定的加性因子 (平均提升 2-3 个 BLEU 点, 如表3最后一列所示)。

放宽容量瓶颈可带来显著的质量提升 在第 4.1 节中, 我们强调了容量瓶颈 对任务干扰的影响, 导致质量下降, 尤其是在高资源语言上。随后我们通过增加每层的专家数量来缓解这一问题, 但这也使所研究的模型的参数 (权重) 数量大幅增加。在此我们考察所谓的容量

瓶颈 是否清晰可见，并探究其在放宽后对模型质量和效率的影响。为此，我们首先考虑三个深度相同（12L）的模型，每层的专家数量依次为 128、512 和 2048。当我们将每层的专家数量从 128 增加到 512（扩大四倍）时，我们观察到模型质量出现大幅跃升，在 100 种语言上的平均 BLEU 分数达到 +3.3。然而，再次按四倍扩展每层的专家数量，从 512 增加到 2048，仅得到 +1.3 的平均 BLEU 分数。尽管质量有显著提升，但增益的下降暗示出现了递减收益。

推测在我们实验设置所采用的特定参数化、语言数量和训练数据量条件下，容量瓶颈预计位于 128 到 512 名专家之间。一旦放宽该瓶颈，模型的深度即可持续扩展，这可通过比较同为 128 名专家的 12 层与 36 层模型看出。有趣的是，如果不放宽容量瓶颈，增加深度的帮助并不大。

增加专家数量可提升质量，尤其有利于高资源任务 另一个有助于解释多任务模型在扩展性方面质量提升的视角，是比较高资源与低资源语言的改进差异。如前所述，低资源语言受益于迁移，而高资源语言则需要额外的容量。接下来我们在固定深度的情况下，考察提高每层专家数的影响。

如图 6 所示，对于 12 层模型，增加专家数量对高资源语言带来更大的增益，而对低资源语言则如先前所示呈现递减收益。对于 36 层模型也观察到类似的模式。尽管增加专家数量会缓解容量瓶颈，但同时由于共享子网络减少，迁移量也会降低。

深层稠密模型在面向低资源任务的正迁移上更优 最后，我们作为对先前实验的一个宽松推论，考察深度对低资源任务的影响。为此，我们将一个在相同数据上用 GPipe 训练的、具有 96 层的稠密模型 T(96L) 纳入分析。我们将 T(96L) 与浅层的专家混合(128E, 12L) 模型进行比较。尽管在大多数高至中等资源的语言上，两种模型之间的差距几乎保持不变，但当进入低资源范围时，这一差距会扩大，且有利于更深的稠密 T(96L) 模型。按照我们先前的论述，随着跨任务的共享子网络比例增加（对于稠密 T(96L) 为 100%），迁移的带宽被最大化，从而相比其浅层对应模型获得更好的质量。还要注意，包含三十七十亿参数的专家混合(36E, 128L) 也能在低资源语言上达到相同的迁移质量。

我们推测，增加深度可能会提高向低资源任务的迁移程度，从而沿该轴获得更好的泛化能力。但我们也想强调，被比较的模型在训练资源需求上存在不成比例的差异。我们再次强调训练效率的重要性，这正是我们接下来研究的主题。

4.5 训练效率

在本节中，我们关注 MoE Transformer 模型的训练效率。迄今为止，我们已经看到了经验证据，表明沿着各个轴向对模型进行扩展可以带来显著的质量提升，并研究了影响改进幅度的因子。为了衡量训练效率，我们首先跟踪为达到某一训练损失所需处理的令牌数量，其次跟踪模型处理某一定量令牌所用的墙钟时间。请注意，我们关注的是训练时间和训练损失¹³，同时变动其他因子；这不同于上一节中分析的测试误差。

更深的模型具有更高的样本效率，用更少的样本更快收敛 已有研究表明，在相同数量的训练样本下，更深的模型在样本效率方面更好，能够取得更低的训练/测试误差 [15, 56]，通常将其归因于过度参数化的加速效应 [1]。我们使用 GShard 与 MoE Transformer（混合专家 Transformer）再次对该假设进行实证检验，并分享针对不仅更深且稀疏激活的模型的权衡。

¹³本节报告的训练损失对应于交叉熵损失，且不包含第2.2节中引入的辅助损失项

为此，我们比较每个模型为达到预设训练损失所需处理的token数量。从表2中我们观察到一个总体趋势：深度提高3倍的 MoE Transformer模型，为达到预设的训练损失阈值，所需的token数量仅为原来的1/2至1/3。例如，与 MoE(128E, 36L) 相比，MoE(128E, 12L) 需要3倍的token数量才能达到0.7的训练交叉熵，（6）对比（5）。对于具有512和2048个专家的模型，我们观察到类似趋势，（4）对比（3）以及（2）对比（1）。

Id	模型	核心	达到交叉熵为 所需的十亿Token数		
			0.7	0.6	0.5
(1)	专家混合(2048E, 36L)	2048	82	175	542
(2)	专家混合(2048E, 12L)	2048	176	484	1780
(3)	专家混合 (512E, 36L)	512	66	170	567
(4)	专家混合 (512E, 12L)	512	141	486	-
(5)	专家混合 (128E, 36L)	128	321	1074	-
(6)	专家混合 (128E, 12L)	128	995	-	-

表2：为达到三种不同的交叉熵损失，模型在训练过程中所见到的 Token 数量。总体趋势是，更深的模型样本效率更高，并且比相当的浅层模型收敛更快。

从表2得到的另一项有趣观察，再次与容量瓶颈的存在有关。比较具有相同深度的模型（5）、（3）和（1），当我们将专家数量从 128 提升到 512 时，为达到 0.7 的训练损失所需的Token 数量显著下降。实际上，这正是我们观察到容量瓶颈所在之处，与第 4.4 节中的假设一致。在这一阶段转变之后，容量充足的模型往往呈现出相似的样本效率特性，如模型（3）和（1）。

最大模型（600B）可在不足 4 天内完成训练并达到最佳质量接下来我们更深入地探讨模型规模与用于训练的墙钟时间之间的相互作用。我们监控使用的 TPU 核心数量、每秒的训练步数、每批次标记总数、TPU 核心年¹⁴，以及以天为单位的实际训练墙钟时间（分别对应表3中的各列）。

我们首先研究我们训练过的最大模型之一，专家混合（2048E, 36L），具有 6000亿参数，模型编号为（1）。使用 2048 个 TPU 核心训练 4 天后，该模型在平均 BLEU 上取得了最佳翻译质量，但训练总计耗费 22.4 个 TPU 年。虽然随着我们扩展模型规模尚未看到质量提升出现平台期，但我们致力于寻找具有成本效益的扩展方案。

表3中的结果再次验证，相比于稠密扩展，使用条件计算进行扩展要实用得多。在使用与（1）相同数量的 TPU 核心的情况下，稠密扩展变体 T(96L) 的训练似乎需要超过十倍的时间（235 个 TPU 核心年），同时在模型质量方面也落后于使用 GShard 训练的模型。

Id	模型	核心	步数 每秒	批次大小 (Token)	TPU 核心 年	训练 时间 (天)	BLEU avg.
(1)	专家混合 (2048E, 36L)	2048	0.72	4M	22.4	4.0	44.3
(2)	专家混合 (2048E, 12L)	2048	2.15	4M	7.5	1.4	41.3
(3)	专家混合(512E (本地专家数), 36L)	512	1.05	1M	15.5	11.0	43.7
(4)	专家混合(512E (本地专家数), 12L)	512	3.28	1M	4.9	3.5	40.0
(5)	专家混合(128E (本地专家数), 36L)	128	0.67	1M	6.1	17.3	39.0
(6)	专家混合(128E (本地专家数), 12L)	128	2.16	1M	1.9	5.4	36.7
*	T(96L)	2048	-	4M	~235.5	~42	36.9

表3：不同专家数量和层数下 MoE 模型的性能。

在用于多语言机器翻译（尤其是 M4）的 MoE Transformer（混合专家 Transformer）应用中，我们对 GShard 进行了基准评测。我们识别出会影响最终结果的变量，例如

¹⁴TPU 核心年简单地通过核心数量与以年为单位的墙钟时间的乘积来度量。

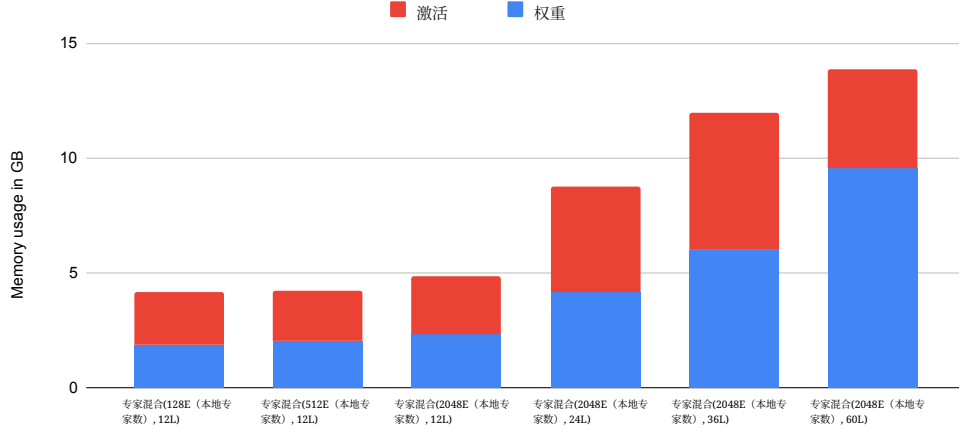


图 7: 每个设备的内存消耗（单位：GB）。

容量瓶颈、正向迁移 和训练效率，并给出了实验结果，以揭示它们之间的相互作用。接下来，我们将深入探讨与 GShard 性能相关的主题，例如内存与运行效率以及通信基准。

5 性能与内存消耗

本节讨论 GShard 在 TPU 平台上实现计算和内存效率的情况。我们的测量与分析表明，当我们增加设备和专家的数量时，每个设备的内存消耗大致保持不变，而步骤时间呈次线性增长；也就是说，当我们将模型从 128 个设备缩放至 2048 个设备（16 倍）时，执行时间仅增加 1.7 倍。我们还为多种分区的算子提供了微基准和分析，可为本论文之外的用例提供参考。

5.1 内存效率与可扩展性

在 GShard 模型中，主要有三类内存使用；在进行 SPMD 划分后，随着专家数量增加，它们在每个设备上的大小都保持不变。

- 复制的权重（例如 Transformer 架构的前馈层）。
- 分布式权重（专家混合前馈层¹⁵）。
- 激活（每一层在前向和反向传播中都会使用的输出）。

图 7 展示了 $O(1)$ 的内存扩展性，显示了不同模型在每个设备上的内存使用分布。在层数固定的情况下，当专家数量增加时，权重内存和激活内存都保持不变。

另一方面，权重内存和激活内存都随层数线性缩放。当内存需求超过每个设备上的可用内存时，基于编译器的再物化会在反向传播中自动重计算部分激活，以降低峰值激活内存。这就是为什么专家混合(2048E, 60L)的激活内存大小小于专家混合(2048E, 36L)。再物化的开销也已优化，例如，对于 36L 和 60L 模型，分别只有 28% 和 34% 的总周期花在重计算上；对于 12L 和 24L，由于它们无需再物化即可放入设备内存，该比例为 0%。

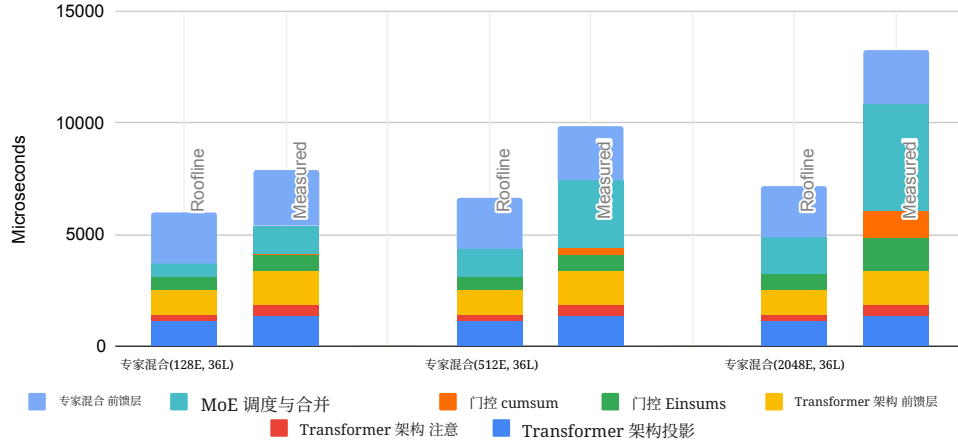


图8: 实测与 roofline 执行时间拆解对比。仅展示前向传播，反向传播具有类似的拆解。
“专家混合 分发和合并”表示使用 AllToAll（全互连）的跨分区通信。

5.2 运行时效率与可扩展性

图8展示了一个 MoE 层及其相邻的 Transformer 架构层的执行时间分解。它还将达到的性能与 Roofline 进行比较，后者的估计假设受计算、内存或通信限制的算子能够达到峰值 FLOPs、内存带宽或互连带宽的 100%。这是一个非常乐观的估计，因为许多算子受多种资源的共同限制。在较小的规模（128 名专家）下，我们的模型可以达到 $> 70\%$ 的 Roofline 性能。当我们将模型扩展到 16 倍（2048 个专家）时，设备时间增加了 1.7 倍，仍可达到 Roofline 性能的 48%。

在分析性能可扩展性之前，我们回顾第 3.1 节中讨论的相关张量维度的尺寸缩放。使用 D 设备时，专家数量 E 和组数 G 均设为 $O(D)$ 。每组专家容量的分数 C 设为 $O(1/D)$ 。该设置无法无限制扩展，因为 C 至少需要为 1，但足以扩展到数千个专家。

Transformer 层与专家混合前馈层 这些是模型的稠密部分，旨在实现 TPU 的峰值利用率。在每个设备上，当我们扩展到更多专家时，这些计算的开销也保持为常数。前馈层和 Transformer 投影主要是大型矩阵乘法，能够很好地利用 TPU 的矩阵单元。在我们的实验中，这些操作达到了 $> 85\%$ 的峰值 FLOPs。注意操作主要由批次矩阵乘法构成，在序列长度较小时受内存带宽所限。因此，在我们的实验中，注意操作仅达到了 $> 30\%$ 的峰值 FLOPs。

门控计算 在图8中，“Gate Einsum”表示算法2中的前两个和最后一个 Einsum。第一个 Einsum 是用于计算每个专家的 Softmax 输入的投影，其代价为 $O(D)$ ，但这只占该层的一小部分。另两个 Einsum 分别用于分发标记和合并专家结果。它们本质上用 one-hot 矩阵实现了 Gather，虽然更昂贵，但其代价是常数 $O(GC) = O(1)$ ，与专家数量无关。当我们将规模从 128 缩放到 2048 个专家（16 倍）时，这些 Einsum 的执行时间增加了约 2 倍。

其余的每个设备上的门控计算涉及许多通用计算，如 argmax （取最大）和 Cumsum，这些计算要么受内存限制，要么本质上是串行的，因此不适合充分利用 TPU。大部分时间花在顺序的 Cumsum 操作上，将表示每个标记所选专家的独热矩阵转换为表示

¹⁵门控投影权重的大小为 $O(E)$ ，可以被划分，但在实践中它们足够小，可以复制，对峰值内存使用的影响可以忽略不计。

每个专家所选标记的独热矩阵。Cumsum 的线性复杂度在图8中得到展示。门控计算的这一部分也具有 $O(D)$ 的代价，但幸运的是，类似于 Softmax 之前的 Einsum，它的常数因子非常小。对于 128 名专家，其执行时间可以忽略不计；对于 2048 个专家，在专家混合和 Transformer 架构层中所占用的总时间不到 10%。

门控中最重要的部分是通信，在图8中显示为“MoE 调度与合并”。这些是 AllToAll（全互连）算子，我们将在第5.3节讨论，其开销为 $O(\sqrt{D})$ 。当专家数量从128增长到2048（增加16倍）时，执行时间约增加3.75倍；它们在专家混合和 Transformer 架构中的执行时间占比从16%上升到36%。

5.3 通信微基准与按算子可扩展性

本节我们测量并分析 SPMD 分区器在基本算子上的性能可扩展性，这些结果可用于指导超出本论文所呈现的专家混合模型之外的用例。

通信原语的性能可扩展性 在专家混合模型中，两个关键的集合通信算子是 AllReduce 和 AllToAll（全互连）。AllReduce 用于累加部分结果，AllToAll（全互连）用于重新分片（第 3.3.2 节）。图9展示了它们在从16到2048个分区时的性能可扩展性。TPU 上的 AllReduce 其执行时间与设备数量无关。图9中的方差来自各个拓扑的具体特性，例如其形状是正方形还是长方形，以及是环形（torus）还是网格（mesh）。

另一方面，随着分区数增加，AllToAll（全互连）的开销会增大，但呈次线性增长。在我们的 2D TPU 集群上，AllToAll（全互连）的开销大致为 $O(\sqrt{D})$ ，其中 D 是分区数。这是因为在每个分区发送的数据量固定时（图9中为 8MB 或 32MB），所有分区发送的总数据量为 $d = O(D)$ 。同时，每条数据平均需要经过 $h = O(\sqrt{D})$ 跳，且网络中共有 $l = O(D)$ 条设备到设备的链路。因此，如果受带宽限制，AllToAll（全互连）的执行时间为

$$t = \frac{dh}{l} = O\left(\frac{D\sqrt{D}}{D}\right) = O(\sqrt{D}).$$

即使受延迟限制，执行时间仍为 $O(h) = O(\sqrt{D})$ 。比较 2048 个分区与 16 个分区，尽管 D 增加了 128 倍，AllToAll（全互连）的执行时间仅增加了 9 倍。这使我们能够使用 resharding 高效实现跨分区分发（图4a）。

AllGather（全收集）and CollectivePermute 更容易分析。AllGather（全收集）的输出是 D 比输入更大，且若固定输入大小，则其通信开销为 $O(D)$ 。CollectivePermute 具有一对一的通信模式，在源-目的对彼此接近的合理设备布局下，固定输入大小时其开销为 $O(1)$ 。

	$O(D)$ 维度	总计 计算	每个分区	
			计算	通信
Add(A (输入令牌矩阵) , A (输入令牌矩阵)) → A (输入令牌矩阵)	A	$O(D)$	$O(1)$	0
Matmul(A (输入令牌矩阵) 批次大小, 批次大小 C (专家容量) → A (输入令牌矩阵) C (专家容量))	B	$O(D)$	$O(1)$	$O(1)$ AR
Matmul(A (输入令牌矩阵) 批次大小, 批次大小 C (专家容量) → A (输入令牌矩阵) C (专家容量))	A	$O(D)$	$O(1)$	0
Matmul(A (输入令牌矩阵) 批次大小, 批次大小 C (专家容量) → A (输入令牌矩阵) C (专家容量))	A, B	$O(D^2)$	$O(D)$	$O(D)$ AG 或 CP
Matmul(A (输入令牌矩阵) 批次大小, 批次大小 C (专家容量) → A (输入令牌矩阵) C (专家容量))	A, C	$O(D^2)$	$O(D)$	$O(D)$ AG 或 CP
Reduce(A (输入令牌矩阵) 批次大小 → A (输入令牌矩阵))	A	$O(D)$	$O(1)$	0
Reduce(A (输入令牌矩阵) 批次大小 → 批次大小)	A	$O(D)$	$O(1)$	$O(1)$ AR
Einsum (SEC, GSM (流式多处理器)) → E (本地专家数) GCM)	G, E (本地专家数) *	$O(D)$	$O(1)$	$O(\sqrt{D})$ AA
卷积 (BIX (专家权重张量) Y, xyIO → BOX (专家权重张量) Y)	X **	$O(D)$	$O(1)$	$O(1)$ CP

表4: 分区算子的可扩展性。通信原语的缩写: AR: AllReduce, AG: AllGather（全收集），CP: CollectivePermute, AA: AllToAll（全互连）。*这是我们模型中的分发 Einsum，其中我们将 C（专家容量）设为 $O(1/D)$ 。**I/O 为输入/输出特征维度，B 为批次大小，X（专家权重张量）/Y 为输入空间维度，x/y 为核的空间维度。

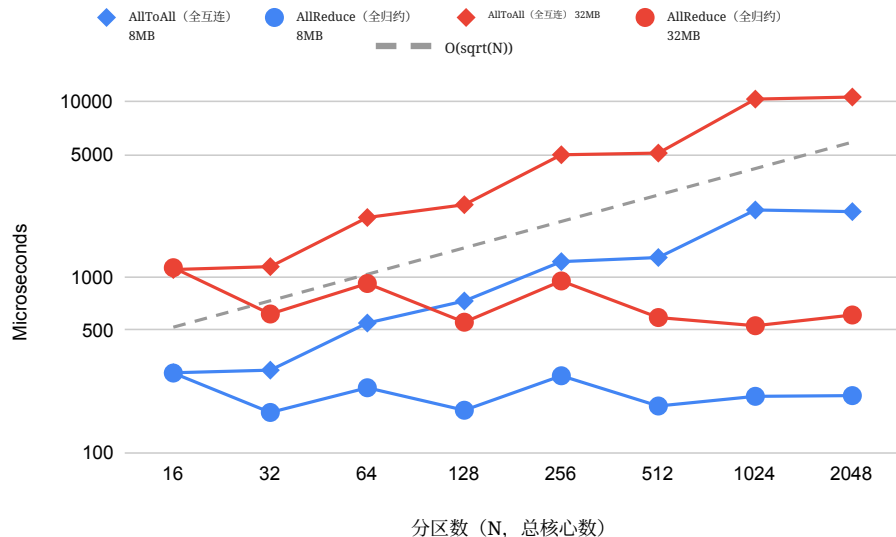


图9: 通信、AllReduce 和 AllToAll (全互连) 的性能扩展性。两轴均为对数刻度。AllReduce 的成本大致为 $O(1)$, 而 AllToAll (全互连) 的成本大致为 $O(\sqrt{D})$, 其中 D 是分区数。我们用 8MB 和 32MB 的数据来测量它们的性能。对于 AllToAll (全互连), 这意味着每个分区最初有 8MB (或 32MB) 数据, 然后将其划分为 D 块, 并将每块发送到不同的接收分区。

分区算子可扩展性 我们在表4中总结了使用 GShard 的常见算子的性能可扩展性。它包含第3.3.2节中的 Einsum/Matmul 示例, 以及其他常见算子, 如 Convolution 和 Reduce。该表包括每个分区上的本地计算, 以及基于我们上述分析所需的通信。

表4中的大多数算子在计算和通信方面都具有次线性可扩展性, 这与我们对专家混合模型的性能测量结果一致。空间分区卷积的 $O(1)$ 扩展性也表明了 GShard 在图像分区方面的高效性 (附录A.4)。

然而, 表4中最后两个 Matmul 算子在每个分区的计算和通信方面具有 $O(D)$ 扩展性, 其操作数的分片方式不匹配。这并非由于分区算法低效, 而是因为完整算子的总计算量非常大 ($O(D^2)$)。针对这些情况可以采用不同的分区策略, 从而产生不同的通信原语: 复制一个操作数将得到 AllGather (全收集) (要求被复制的操作数能够放入设备内存), 而在循环中进行切片 (图4c) 将得到 CollectivePermute。

6 相关工作

神经网络 深度学习模型在推进人工智能各子领域方面非常成功。多年来, 这些领域一直在持续报告新的最先进成果, 采用多种模型架构用于计算机视觉任务 [57, 58, 7], 用于自然语言理解任务 [59, 60, 61], 用于语音识别与合成任务 [62, 63, 64, 65, 66]。最近, 基于注意的 Transformer 模型进一步提升了这些领域的最先进水平 [10, 4]。

模型扩展性 学术研究和行业应用都观察到, 在足够大的数据集上以及针对复杂任务时, 更大的神经网络往往表现更好。在同一模型家族内, 单纯把网络做得更宽或更深, 往往在经验上能提升模型质量。例如, 更深的 ResNets 表现更好 [8], 更大的 Transformer 模型取得了更好的翻译质量 [10], 拥有更大词表, 或更大的嵌入或特征交叉的模型也表现更佳 [14, 13]。在

不同的模型家族中，也观察到，更大的模型、具有更大模型容量者，不仅能更好地拟合训练数据，在测试时也具有更好的泛化能力 [67, 68, 15]。这一观察促使许多研究致力于构建远大于深度学习研究模型或生产模型通常规模的神经网络。Shazeer 等表明，一个使用 Mixture-of-Experts 专家层的、具有 690 亿参数的循环语言模型，在 One Billion Word (LM1B) 基准上的测试困惑度显著更低 [16]。Brown 等表明，一个非稀疏的、具有 1750 亿参数的模型，能够在若干下游自然语言处理任务上展现出高度准确的小样本性能。

硬件 神经网络需要不可忽视的计算能力。为了满足这种需求，用于神经网络训练和推理的专用硬件（芯片和联网机器）可以追溯到25年前 [69]。自2000年代末以来，研究者开始利用GPU加速神经网络 [70, 57, 71]。最近，业界也在构建更为专用的硬件系统上投入巨资，以追求更具成本效益的神经网络硬件 [72]。由于神经网络的核心计算（各种乘法求和形式：卷积、矩阵乘法、einsum）是高度可并行的数值计算，这些芯片配备了大量的浮点处理单元（FPU）。因此，这些专门设计的硬件的计算能力显著增长。据报道，过去短短4年间GPU每FLOPs的价格下降了一个数量级（约十倍） [73]，而过去12年间每瓦特的FLOPs提高了2个数量级 [74]。广泛可得、低成本的计算能力是神经网络成功的一个重要推动因素。

软件 支持神经网络的软件系统随着底层硬件 [75, 76, 21, 77]的进步而共同演进。尽管这些加速器是高度并行的计算机器，但直接对其编程要困难得多。框架让构建神经网络变得更容易，并为实践者抽象掉了许多硬件特定的细节。它们又依赖更低层的库来高效驱动专用硬件（加速器）。例如，CUDA [78]用于英伟达的 GPU，或 XLA 用于 Google 的 TPU [28]。这些低层库对于在这类专用硬件上实现高效率至关重要。

模型训练与推理中的并行 现代神经网络在训练和推理中广泛使用机器集群，每台机器都配备了若干加速器。数据并行 [57] 是最常用的方法，并被主流框架 (TensorFlow [21], PyTorch [22], JAX [79, 80]) 所支持，其中，各设备使用不同的输入数据运行相同的程序，并在更新权重之前合并各自的本地梯度。另一方面，模型并行则将计算划分到超出输入批次的范围，这是构建大规模模型所必需的。例如，流水化 [15, 24] 将大型模型的层拆分为多个阶段，而算子级划分 [23, 81] 则把单个算子拆分为更小的并行算子。GShard 使用了一种算子级划分，将我们的模型缩放到大并行专家。

自动并行 由于在分布式异构环境中编程具有挑战性，尤其对高层实践者而言，深度学习框架尝试减轻用户指定分布式计算方式的负担。例如，TensorFlow [21] 支持数据并行，以及通过按每个节点的设备分配进行图划分的基础模型并行。网络 TensorFlow [23] 通过在 TensorFlow 之上的 Python 库中重写计算，以 SPMD 风格按算子划分，帮助用户构建大规模模型；相比之下，我们的方法在编译器中基于轻量级注解对图进行划分，而无需用户重写模型。FlexFlow [81] 使用自动搜索来发现图中算子的最优划分，以获得更好的性能；其侧重于确定划分策略，而我们的 SPMD 划分器侧重于将带注解的图进行变换的机制。权重更新分片 [82]是另一种基于 XLA 的自动并行化变换，主要关注 TPU 集群的性能优化，在概念上可视为 GShard 的一个特例。Zero [83] 提出了一组优化，通过分别划分权重、激活和优化器状态来减少并行训练设备中的内存冗余，并能够将模型缩放到 1700 亿参数；相比之下，GShard 在不区分这些张量这一点上更为通用，所有这些特定的划分技术都可以通过简单地添加注解来支持，使我们能够缩放到超过 1 万亿参数并探索更多设计选择。

条件计算与机器翻译 条件计算 [25, 16, 26, 27]认为，应通过激活一个依赖于输入的子-

网络，在网络内部对样本进行路由。路由取决于（或受制于）某些准则，并且在不失一般性的情况下，可以是以下任意一种：样本的估计难度 [84], 可用的计算预算[26, 27], 或更一般地，稀疏性诱导的专家混合所学习得到的准则 [16]。我们扩展了稀疏门控的专家混合 [16]，由于其灵活性和易于扩展，将其应用于最先进的神经序列模型 Transformer 模型 [10]，以满足训练效率要求。

7 结论

本文介绍了 GShard，这是一种可自动对大规模计算进行分片的深度学习模块。GShard 仅需在用户模型代码中添加轻量级分片注解即可运行，并提供易用且灵活的 API，用于扩展巨型神经网络。我们将 GShard 应用于带有稀疏门控专家混合层的 Transformer 架构（MoE Transformer（混合专家 Transformer）），并证明一个拥有 6000 亿参数的多语言神经机器翻译模型可以在 4 天内高效完成训练：使用单一模型将 100 种语言翻译为英语时，其性能和质量均优于既有方法。除了显著更好的翻译质量外，使用 GShard 训练的 MoE Transformer 模型在训练效率方面也表现出色：训练成本为 22 个 TPU v3 核心年，而训练全部 100 个双语 Transformer 基线模型则需要 29 个 TPU 年。本文给出的实证结果证实，通过利用条件计算来扩展模型不仅能提升真实世界机器学习应用的质量，而且在训练过程中仍然实用且样本效率高。我们提出的方法在可扩展性/成本的权衡上具有优势，并降低了为扩展巨型神经网络而对模型特定框架或工具的需求。总体而言，这些结果有助于阐明一条切实可行的神经网络扩展路径，以实现更好的模型质量。

我们从研究获得了若干经验。我们的结果表明，逐步扩展神经网络会带来持续的质量增益，这验证了随着我们扩展模型，质量提升尚未进入平台期。虽然本文的结果巩固了这样一个结论：模型扩展是深度学习从业者工具箱中的必备手段，但我们也呼吁从业者重视训练效率。为此，我们识别了影响训练效率的因素，并展示了它们对下游任务质量的影响。我们展示了采用条件计算构建的神经网络如何在缩放与计算成本之间取得更有利的权衡。在实践中，这些关键的设计决策使我们能够将实验迭代周期从数月或数周缩短到仅需数天，就能训练具有万亿参数量级的模型。

此外，设置一个将模型描述与并行化实现分离的恰当抽象层，可以让模型开发者专注于网络实现，将计算图的自动划分和在所有设备上并行运行的程序生成交给 GShard。我们发现，生成一个足够通用、能够在底层所有并行设备上表达计算的单个程序，是以可扩展方式进行编译的关键。传统做法是为不同分区生成多个专用程序，但当扩展到数千个分区时，会导致编译时间爆炸式增长。为应对这种复杂性，我们引入了基于 SPMD 分片的多项编译器革新，使得任何张量维度都可以被划分。作为要点，我们强调，模型扩展与训练效率应当齐头并进；并且，诸如条件计算之类的算法改进结合易用的接口，能够高效利用大规模计算能力。

最后，我们的实验结果从经验上表明，仅仅统计参数数量并不总是与在缩放条件下模型的有效容量相关 [85, 86]。对模型的比较还应考虑问题的本质，即如同我们的情况，极大规模的多任务设置下，不同任务之间的训练数据严重不平衡；并控制影响网络不同运行模式的因子，例如容量瓶颈与正迁移。

致谢

我们谨向 Google Brain 和 Translate 团队表示感谢，感谢他们提出的有益意见和富有洞见的讨论；也感谢整个 XLA 和 Lingvo 开发团队对本项目作出的基础性贡献。特别感谢 Youlong Cheng、Naveen Arivazhagan、Ankur Bapna、Ruoming Pang、Yonghui Wu、Yuan Cao、David Majnemer、James Molloy、Peter Hawkins、Blake Hechtman、Mark Heffernan，

Dimitris Vardoulakis, Tamas Berghammer, Marco Cornero, Cong Liu, Tong Shen, Hongjun Choi, Jianwei Xie, Sneha Kudugunta, and Macduff Hughes.

References

- [1] Sanjeev Arora, Nadav Cohen, and Elad Hazan. On the optimization of deep networks: Implicit acceleration by overparameterization. *arXiv preprint arXiv:1802.06509*, 2018.
- [2] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635*, 2018.
- [3] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [5] Dhruv Mahajan, Ross Girshick, Vignesh Ramanathan, Kaiming He, Manohar Paluri, Yixuan Li, Ashwin Bharambe, and Laurens van der Maaten. Exploring the limits of weakly supervised pretraining. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 181–196, 2018.
- [6] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.
- [7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *European conference on computer vision*, pages 630–645. Springer, 2016.
- [9] Golnaz Ghiasi, Tsung-Yi Lin, and Quoc V Le. Nas-fpn: Learning scalable feature pyramid architecture for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7036–7045, 2019.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017.
- [11] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2019.
- [12] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.
- [13] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale, 2019.
- [14] Naveen Arivazhagan, Ankur Bapna, Orhan Firat, Dmitry Lepikhin, Melvin Johnson, Maxim Krikun, Mia Xu Chen, Yuan Cao, George Foster, Colin Cherry, Wolfgang Macherey, Zhifeng Chen, and Yonghui Wu. Massively multilingual neural machine translation in the wild: Findings and challenges, 2019.
- [15] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in Neural Information Processing Systems 32*, pages 103–112, 2019.

- [16] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [17] Madhu S. Advani and Andrew M. Saxe. High-dimensional dynamics of generalization error in neural networks, 2017.
- [18] Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md. Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. Deep learning scaling is predictable, empirically, 2017.
- [19] Joel Hestness, Newsha Ardalani, and Gregory Diamos. Beyond human-level accuracy. *Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming*, Feb 2019.
- [20] Mario Geiger, Arthur Jacot, Stefano Spigler, Franck Gabriel, Levent Sagun, Stéphane d’Ascoli, Giulio Biroli, Clément Hongler, and Matthieu Wyart. Scaling description of generalization with number of parameters in deep learning. *Journal of Statistical Mechanics: Theory and Experiment*, 2020(2):023401, Feb 2020.
- [21] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. Tensorflow: a system for large-scale machine learning. In *OSDI*, volume 16, pages 265–283, 2016.
- [22] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017.
- [23] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, HyukJoong Lee, Mingsheng Hong, Cliff Young, et al. Mesh-tensorflow: Deep learning for supercomputers. In *Advances in Neural Information Processing Systems*, pages 10414–10423, 2018.
- [24] Aaron Harlap, Deepak Narayanan, Amar Phanishayee, Vivek Seshadri, Nikhil Devanur, Greg Ganger, and Phil Gibbons. Pipedream: Fast and efficient pipeline parallel dnn training. *arXiv preprint arXiv:1806.03377*, 2018.
- [25] Emmanuel Bengio, Pierre-Luc Bacon, Joelle Pineau, and Doina Precup. Conditional computation in neural networks for faster models, 2015.
- [26] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. *ArXiv*, abs/1910.10073, 2020.
- [27] Ankur Bapna, Naveen Arivazhagan, and Orhan Firat. Controlling computation versus quality for neural sequence models, 2020.
- [28] XLA: Optimizing Compiler for TensorFlow. <https://www.tensorflow.org/xla>, 2019. Online; accessed 1 June 2020.
- [29] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In *ICML*, 2010.
- [30] Albert Einstein. Die grundlage der allgemeinen relativitätstheorie. In *Das Relativitätssprinzip*, pages 81–124. Springer, 1923.
- [31] Jonathan Shen, Patrick Nguyen, Yonghui Wu, Zhifeng Chen, Mia Xu Chen, Ye Jia, Anjuli Kannan, Tara Sainath, Yuan Cao, Chung-Cheng Chiu, et al. Lingvo: a modular and scalable framework for sequence-to-sequence modeling. *arXiv preprint arXiv:1902.08295*, 2019.
- [32] Youlong Cheng, HyukJoong Lee, and Tamas Berghammer. Train ML models on large images and 3D volumes with spatial partitioning on Cloud TPUs. <https://cloud.google.com/blog/products/ai-machine-learning/train-ml-models-on-large-images-and-3d-volumes-with-spatial-partitioning-on-cloud-tpus>, 2019. Online; accessed 12 June 2020.

- [33] ONNX: Open Neural Network Exchange. <https://github.com/onnx/onnx>, 2019. Online; accessed 1 June 2020.
- [34] Jared Roesch, Steven Lyubomirsky, Logan Weber, Josh Pollock, Marisa Kirisame, Tianqi Chen, and Zachary Tatlock. Relay: a new ir for machine learning frameworks. *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages - MAPL 2018*, 2018.
- [35] Nadav Rotem, Jordan Fix, Saleem Abdulrasool, Garret Catron, Summer Deng, Roman Dzhabarov, Nick Gibson, James Hegeman, Meghan Lele, Roman Levenstein, Jack Montgomery, Bert Maher, Satish Nadathur, Jakob Olesen, Jongsoo Park, Artem Rakhov, Misha Smelyanskiy, and Man Wang. Glow: Graph lowering compiler techniques for neural networks, 2018.
- [36] MPI Forum. MPI: A Message-Passing Interface Standard. Version 2.2, September 4th 2009. available at: <http://www.mpi-forum.org> (Dec. 2009).
- [37] Minsik Cho, Ulrich Finkler, and David Kung. BlueConnect: Decomposing All-Reduce for Deep Learning on Heterogeneous Network Hierarchy. In *Proceedings of the Conference on Systems and Machine Learning (SysML)*, Palo Alto, CA, 2019.
- [38] Lynn Elliot Cannon. *A Cellular Computer to Implement the Kalman Filter Algorithm*. PhD thesis, USA, 1969. AAI7010025.
- [39] Orhan Firat, Kyunghyun Cho, and Yoshua Bengio. Multi-way, multilingual neural machine translation with a shared attention mechanism. *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2016.
- [40] Melvin Johnson, Mike Schuster, Quoc V. Le, Maxim Krikun, Yonghui Wu, Zhifeng Chen, Nikhil Thorat, Fernanda Viégas, Martin Wattenberg, Greg Corrado, and et al. Google’s multilingual neural machine translation system: Enabling zero-shot translation. *Transactions of the Association for Computational Linguistics*, 5:339–351, Dec 2017.
- [41] Roei Aharoni, Melvin Johnson, and Orhan Firat. Massively multilingual neural machine translation. *CoRR*, abs/1903.00089, 2019.
- [42] Exploring massively multilingual, massive neural machine translation. <https://ai.googleblog.com/2019/10/exploring-massively-multilingual.html>. Accessed: 2020-06-05.
- [43] Recent advances in google translate. <https://ai.googleblog.com/2020/06/recent-advances-in-google-translate.html>. Accessed: 2020-06-05.
- [44] Timothy T Baldwin and J Kevin Ford. Transfer of training: A review and directions for future research. *Personnel psychology*, 41(1):63–105, 1988.
- [45] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation, 2013.
- [46] Andrew Davis and Itamar Arel. Low-rank approximations for conditional feedforward computation in deep neural networks, 2013.
- [47] Jakob Uszkoreit, Jay M. Ponte, Ashok C. Popat, and Moshe Dubiner. Large scale parallel document mining for machine translation. In *Proceedings of the 23rd International Conference on Computational Linguistics, COLING ’10*, page 1101–1109, USA, 2010. Association for Computational Linguistics.
- [48] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting on association for computational linguistics*, pages 311–318. Association for Computational Linguistics, 2002.
- [49] Ankur Bapna, Mia Chen, Orhan Firat, Yuan Cao, and Yonghui Wu. Training deeper neural machine translation models with transparent attention. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018.

- [50] Kazuki Irie, Albert Zeyer, Ralf Schlüter, and Hermann Ney. Language modeling with deep transformers. *Interspeech 2019*, Sep 2019.
- [51] Qiang Wang, Bei Li, Tong Xiao, Jingbo Zhu, Changliang Li, Derek F. Wong, and Lidia S. Chao. Learning deep transformer models for machine translation. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [52] David R. So, Chen Liang, and Quoc V. Le. The evolved transformer, 2019.
- [53] Using bfloat16 with TensorFlow models. <https://cloud.google.com/tpu/docs/bfloat16>, 2020. Online; accessed 12 June 2020.
- [54] Heng-Tze Cheng, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, Hemal Shah, Levent Koc, Jeremiah Harmsen, and et al. Wide and deep learning for recommender systems. *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems - DLRS 2016*, 2016.
- [55] Andrew K. Lampinen and Surya Ganguli. An analytic theory of generalization dynamics and transfer learning in deep linear networks, 2018.
- [56] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using gpu model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [57] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pages 1097–1105, 2012.
- [58] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015.
- [59] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In *Advances in neural information processing systems*, pages 3104–3112, 2014.
- [60] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.
- [61] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- [62] Geoffrey Hinton, Li Deng, Dong Yu, George E Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara N Sainath, et al. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal processing magazine*, 29(6):82–97, 2012.
- [63] William Chan, Navdeep Jaitly, Quoc Le, and Oriol Vinyals. Listen, attend and spell: A neural network for large vocabulary conversational speech recognition. In *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4960–4964. IEEE, 2016.
- [64] Chung-Cheng Chiu, Tara N Sainath, Yonghui Wu, Rohit Prabhavalkar, Patrick Nguyen, Zhifeng Chen, Anjuli Kannan, Ron J Weiss, Kanishka Rao, Ekaterina Gonina, et al. State-of-the-art speech recognition with sequence-to-sequence models. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4774–4778. IEEE, 2018.
- [65] Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.

- [66] Jonathan Shen, Ruoming Pang, Ron J Weiss, Mike Schuster, Navdeep Jaitly, Zongheng Yang, Zhifeng Chen, Yu Zhang, Yuxuan Wang, Rj Skerrv-Ryan, et al. Natural tts synthesis by conditioning wavenet on mel spectrogram predictions. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4779–4783. IEEE, 2018.
- [67] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. 2017.
- [68] Behnam Neyshabur, Srinadh Bhojanapalli, David McAllester, and Nathan Srebro. Exploring generalization in deep learning, 2017.
- [69] Paolo Ienne, Thierry Cornu, and Gary Kuhn. Special-purpose digital hardware for neural networks: An architectural survey. *Journal of VLSI signal processing systems for signal, image and video technology*, 13(1):5–25, 1996.
- [70] Rajat Raina, Anand Madhavan, and Andrew Y Ng. Large-scale deep unsupervised learning using graphics processors. In *Proceedings of the 26th annual international conference on machine learning*, pages 873–880, 2009.
- [71] Dan Claudiu Cireşan, Ueli Meier, Luca Maria Gambardella, and Jürgen Schmidhuber. Deep, big, simple neural nets for handwritten digit recognition. *Neural computation*, 22(12):3207–3220, 2010.
- [72] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*, pages 1–12, 2017.
- [73] 2019 recent trends in GPU price per FLOPS. <https://aiimpacts.org/2019-recent-trends-in-gpu-price-per-flops/>. Accessed: 2020-06-05.
- [74] Yifan Sun, Nicolas Bohm Agostini, Shi Dong, and David Kaeli. Summarizing cpu and gpu design trends with product data. *arXiv preprint arXiv:1911.11313*, 2019.
- [75] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc’aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, et al. Large scale distributed deep networks. In *Advances in neural information processing systems*, pages 1223–1231, 2012.
- [76] Frédéric Bastien, Pascal Lamblin, Razvan Pascanu, James Bergstra, Ian Goodfellow, Arnaud Bergeron, Nicolas Bouchard, David Warde-Farley, and Yoshua Bengio. Theano: new features and speed improvements. *arXiv preprint arXiv:1211.5590*, 2012.
- [77] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017.
- [78] John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. Scalable parallel programming with cuda. *Queue*, 6(2):40–53, 2008.
- [79] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, and Skye Wanderman-Milne. JAX: composable transformations of Python+NumPy programs. 2018.
- [80] Roy Frostig, Matthew Johnson, and Chris Leary. Compiling machine learning programs via high-level tracing. In *Machine Learning and Systems (MLSys)*, 2018.
- [81] Zhihao Jia, Matei Zaharia, and Alex Aiken. Beyond Data and Model Parallelism for Deep Neural Networks. In *Proceedings of the Conference on Systems and Machine Learning (SysML)*, Palo Alto, CA, 2019.
- [82] Yuanzhong Xu, HyoukJoong Lee, Dehao Chen, Hongjun Choi, Blake Hechtman, and Shibo Wang. Automatic cross-replica sharding of weight update in data-parallel training, 2020.

- [83] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimization towards training a trillion parameter models. *arXiv preprint arXiv:1910.02054*, 2019.
- [84] Loren Lugosch, Derek Nowrouzezahrai, and Brett H. Meyer. Surprisal-triggered conditional computation with neural networks, 2020.
- [85] Chunyuan Li, Heerad Farkhoor, Rosanne Liu, and Jason Yosinski. Measuring the intrinsic dimension of objective landscapes, 2018.
- [86] Wesley J. Maddox, Gregory Benton, and Andrew Gordon Wilson. Rethinking parameter counting in deep models: Effective dimensionality revisited, 2020.
- [87] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- [88] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. *ArXiv*, abs/1804.04235, 2018.
- [89] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *EMNLP*, 2018.

A Appendix

A.1 Decoding with Flat Beam Search

During decoding, we use beam search with length normalization similar to [61]. Decoding is auto-regressive and generates the target sequence one token at a time, so for an output of length m the decoder layer stack is executed m times, sequentially. In particular for each decoder MoE layer there are dispatch/combine operations, which require cross-device communication. Inference utilizes same cluster with same number of devices as training.

During beam search we flatten the beam hypotheses into a single sequence which contains all underlying tokens interleaved, and we modify decoder self-attention mask so that each hypothesis only has attention to appropriate positions in the joint flat sequence. We apply the same transformation to key/value tensors maintained by each decoder self-attention layer. This allows us to avoid reordering previously computed attention key/values after each beam expansion. Instead, we only reorder the 0/1 mask representing the current active hypotheses. However, attention becomes k times longer.

This trade-off can be positive or negative depending on implementation details. As explained in [87], memory bandwidth limits are important for incremental decoding with Transformer models. From this point of view, by flattening the beam we replace two operations with low compute/memory ratio (attention dot product and key/value reordering) with a single operation with a slightly higher compute/memory ratio (attention dot product over a longer sequence with more keys), but with the same total amount of memory it has to access.

A.2 Machine Translation Experiments Details

In our Machine Translation experiments MoE Transformer models shared *a)* 1024 Transformer model dimension *b)* 8192 Feed Forward and MoE hidden dimension; *c)* 16 heads in multi-head attention; *d)* 128 attention key and value dimension; and *e)* 0.1 input, residual and attention dropout rate.

We used the Adafactor [88] optimizer with *a)* factored second-moment estimation; *b)* first moment decay $\beta_1 = 0.0$; *c)* second moment decay $\beta_2 = 0.99$ with $1 - t^{-0.8}$ schedule; *d)* update clipping threshold of 1.0; and *e)* 1.0 learning rate with square root decay after 10k training steps.

We used SentencePiece [89] subword tokenizer with a single multilingual vocabulary for source-side spanning 102 languages of size 64000, and English-only target-side vocabulary of size 32000.

A.3 General Sharding API

In addition to the two common APIs (`replicate()` and `split()`) for sharding listed in Section 3.2, users or the compiler may use a more advanced sharding strategy to minimize data transfers.

shard(tensor, device_assignment) annotates `tensor` to be partitioned with the provided device assignment, and returns the annotated tensor. We use *device assignment*, a multi-dimensional integer array, to represent how the split is done. `device_assignment` has the same rank as the data tensor; its element count is the total number of partitions, and each element is the ID of the device that occupies the corresponding data slice. For example, a 3D tensor with shape $[3, 16, 64]$ with device assignment shape $[1, 2, 4]$ will have partition shape $[3, 8, 16]$, and the order of elements in *device assignment* determines which slice each partition occupies.

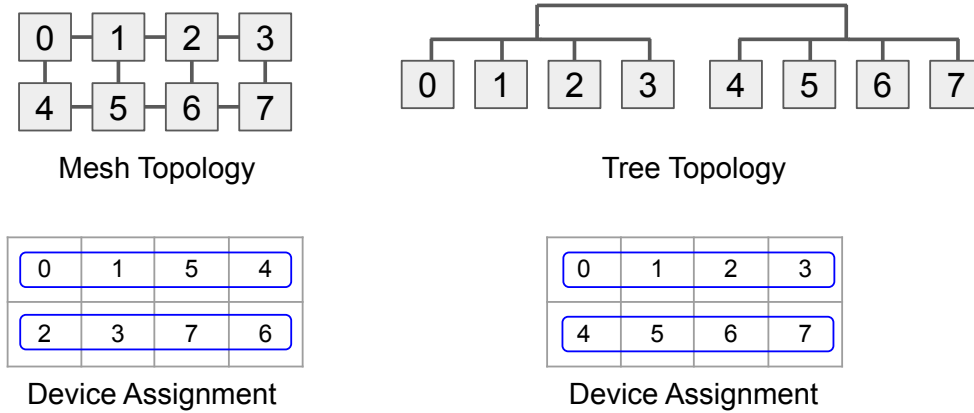


Figure 10: An example of two different *device assignments* based on the device topology. A 2D tensor is split by 2x4 partitions and the communication pattern is between partitions along the rows of the tensor. The numbers represent device ids.

Since data movement across devices critically affects the parallel execution performance, it is important to consider the target device topology as well as the communication between partitions of the tensor when assigning device ids in the *device assignment* for maximum performance. Figure 10 shows two different *device assignments* based on the device topology and the row-wise communication pattern on the tensor.

A.4 SPMD Partitioning for Convolution and Window-Based Operators

GShard is able to partition spatial dimensions in convolutions, and general enough to support use cases like giant images [32]. To spatially shard a convolutional layer, we can use the sharding API in the following way.

```
# Partition input images [N,C,H,W] along W spatial dimension
inputs = split(inputs, 3, D)
# Replicate the kernel
kernel = replicate(kernel)
conv = conv2d(inputs, kernel)
...
```

GShard will then propagate the sharding on the spatial dimension to other layers and the backward pass. The rest of section discusses the specific complexity to partition Convolution and similar operators. There are several window-based operations (e.g., Convolution, ReduceWindow), and they all require some type of halo exchange since data may be shared between windows. We use the `CollectivePermute` operator to exchange halo data between partitions, but one complication is that the halo size may be different across partitions whereas `CollectivePermute` needs to be statically shaped.

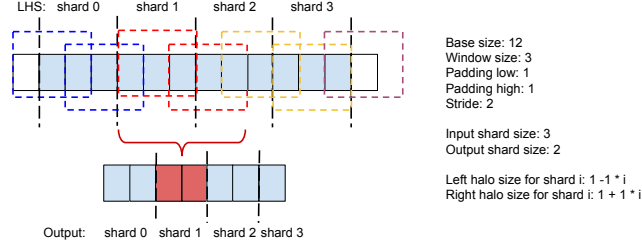


Figure 11: Convolution with non-constant halo size.

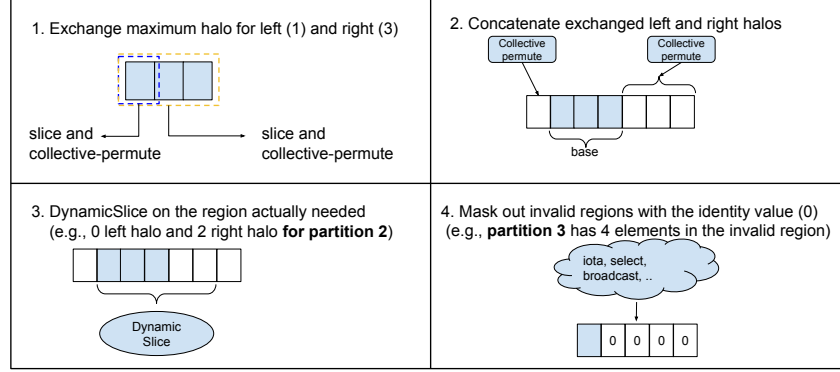


Figure 12: Sequence of operations for a general halo exchange.

We first introduce the window configurations that the SPMD partitioner has to consider. Each spatial dimension in the convolution has the following set of configurations.

- **Stride** is the distance (in number of elements) that the window moves to produce the next output element.
- **Low/high padding** is the number of elements padded to the low/high end of the dimension in LHS (base).
- **Base dilation** is the dilation factor of the LHS, i.e., one plus the number of elements padded between every element (excluding low/high padding). No base dilation means the value is set to 1.
- **Window dilation** is one plus the number of elements padded between every element in the RHS (window).

Non-constant halo size. We demonstrate that non-constant halo size is common using a simple example, which does not have dilation. Figure 11 shows a 4-way partitioned convolution, where the right halo sizes for the partitions are (1, 2, 3, 4) and can be expressed as a linear function of the partition ID: $partition_id + 1$. Partition 1 is in charge of generating 2 output elements (red cells), which means that the partition needs to get 0 elements from Partition 0, and 2 elements from Partition 2 (area covered by two dotted red windows).

Figure 12 describes the sequence of operations for a general halo exchange. First, we calculate the maximum size of left and right halo across partitions and perform the halo exchange of the maximum size (Steps 1 and 2). Since some partitions may have excessive halos than needed, we use DynamicSlice (based on the partition ID) to slice off the valid region for the current partition (Step 3). Finally, some partitions may include garbage values (e.g., halos from out-of-range input data), so we apply masking as described in Section 3.3.3.

Base dilation. Base dilation adds additional complexities to halo exchange, since the offset of each partition may be positioned at the dilation holes, and also low/high padding is applied after dilation, which makes the edges have different behavior than the interior elements. We handle base dilation in 3 cases (Figure 13).

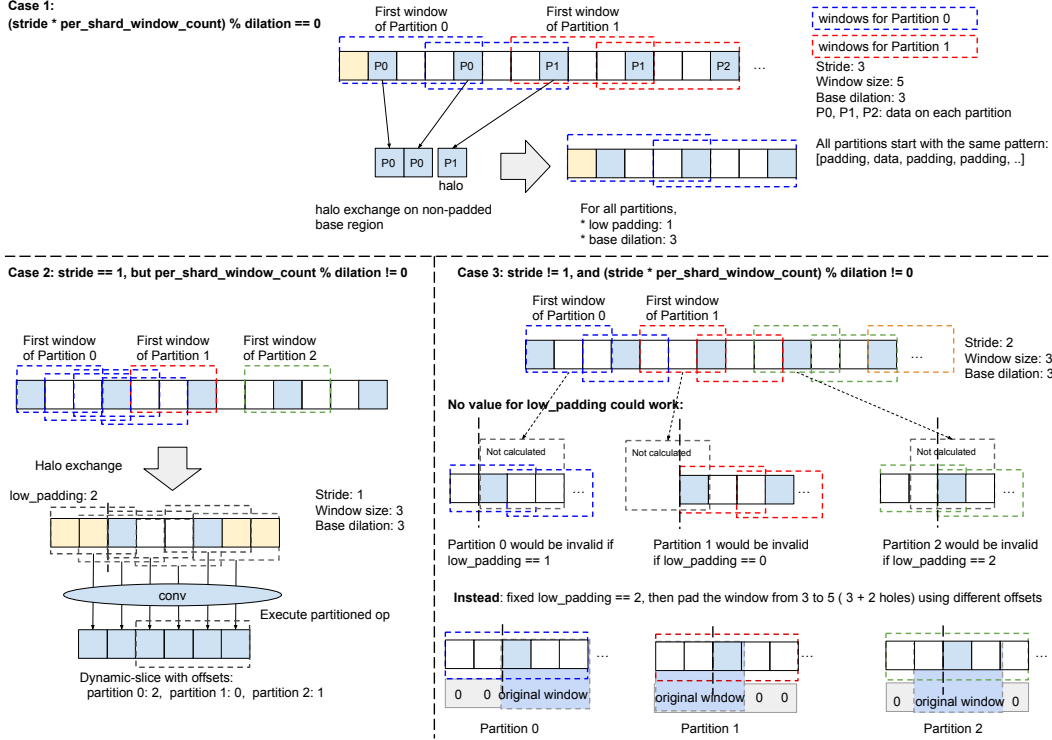


Figure 13: Partitioned convolution with base dilation.

- $\text{stride} \times \text{per_shard_window_count}$ is divisible by dilation , where $\text{per_shard_window_count}$ is the number of windows to be processed by each partition (i.e., the number of output elements for each partition). This condition guarantees that all partitions start with the same number of (interior or low) padding elements before the first data element in the LHS, so that we can use the same low padding. Halo exchange occurs on the non-dilated/non-padded base region, and the limit index of required data for Partition i can be calculated as below.

$$\frac{\text{stride} \times \text{per_shard_window_count} \times i + \text{window_size} - \text{low_pad} + \text{dilation} - 1}{\text{dilation}},$$

which determines the right halo size. Because $\text{stride} \times \text{per_shard_window_count}$ is divisible by dilation , it can be simplified as $a \times i + b$, where a and b are both constants.

- $\text{stride} == 1$ but $\text{per_shard_window_count}$ is not divisible by dilation . In this case, the low padding on different partitions are different, but it is a static configuration in windowed operations, which can't be specialized for each partition for SPMD execution. Using Pad and DynamicSlice on the operand also would not work, because those operators would be applied before dilation, so everything would be multiplied by the dilation factor. Fortunately, with $\text{stride} == 1$, all positions on the padded and dilated base region are valid window starts, and we can use the maximum low padding on all partitions to ensure that each partition calculates all required windows, then do a DynamicSlice on the output of the partitioned windowed operator to remove unnecessary data. The limit index of required data on the non-padded base region for Partition i is same as before,

$$\frac{\text{per_shard_window_count} \times i + \text{window_size} - \text{low_pad} + \text{dilation} - 1}{\text{dilation}},$$

but cannot be simplified to $a \times i + b$.

- $\text{stride} \neq 1$ and $\text{stride} \times \text{per_shard_window_count}$ is not divisible by dilation . If neither of the above conditions are true, different partitions could start with different number of padding elements, and not all offsets are valid window starts. Consider the last example in

Figure 13. Whatever low padding we chose, some partition will be invalid, because the valid windows could be skipped since $stride \neq 1$. A solution to this problem is to pad the window in addition to padding the base area. We can use the maximum low padding required by the partitions on the base area, then increase the window size by that low padding amount. However, the low and high padding amounts on the window vary on different partitions, which can be implemented by a `Pad` followed by a `DynamicSlice`. The window padding is used to mask off the unaligned elements in the base area, so that the start of the non-padding window element will be aligned with the desired start in the base area for each partition.

Window dilation. If the RHS is replicated, window dilation only affects the effective window size when partitioning the operator based on its LHS. If the dilated RHS is also partitioned, which typically occurs in the gradient computation of strided convolutions, handling window dilation is still simpler than handling base dilation, because there is no low/high padding on the RHS. We skip the details of the implementation.